Exp No: 1	Study of various Network commands used in Linux and Windows

AIM:

To study and practise the various network commands used in Linux and Windows operating systems.

ALGORITHM / PROCEDURE:

1. Open the Terminal in Linux and the Command Prompt (`cmd`) in Windows.
2. Use `hostname` to see the name of the computer.
3. Use `ifconfig` (Linux) and `ipconfig` (Windows) to see the IP address, subnet mask and MAC address.
4. Use `ping` to check whether the remote system is reachable. Note the packet loss and the time taken.
5. Use `traceroute` (Linux) and `tracert` (Windows) to see the path taken by the packets.
6. Use `nslookup` and `dig` to find the IP address of a domain name.
7. Use `netstat` to see the active connections and open ports.
8. Use `arp -a` to see the IP address to MAC address table.
9. Use `route` (Linux) and `route print` (Windows) to see the routing table.
10. Note down the output of each command.

Commands used:

Purpose	Linux	Windows
Host name	`hostname`	`hostname`
IP configuration	`ifconfig / ip addr`	`ipconfig /all`
Check connectivity	`ping -c 4 google.com`	`ping google.com`
Trace the route	`traceroute google.com`	`tracert google.com`
DNS lookup	`nslookup / dig`	`nslookup`
Active connections	`netstat -tuln`	`netstat -an`
ARP table	`arp -a`	`arp -a`
Routing table	`route -n`	`route print`
MAC address	`ip link`	`getmac`

DIAGRAM:

Fig 1: Network commands and the layers they work in

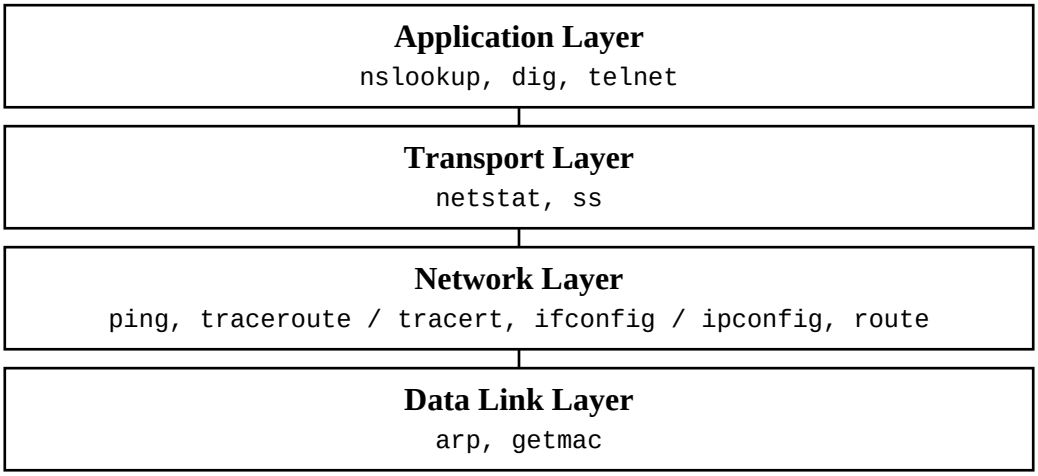
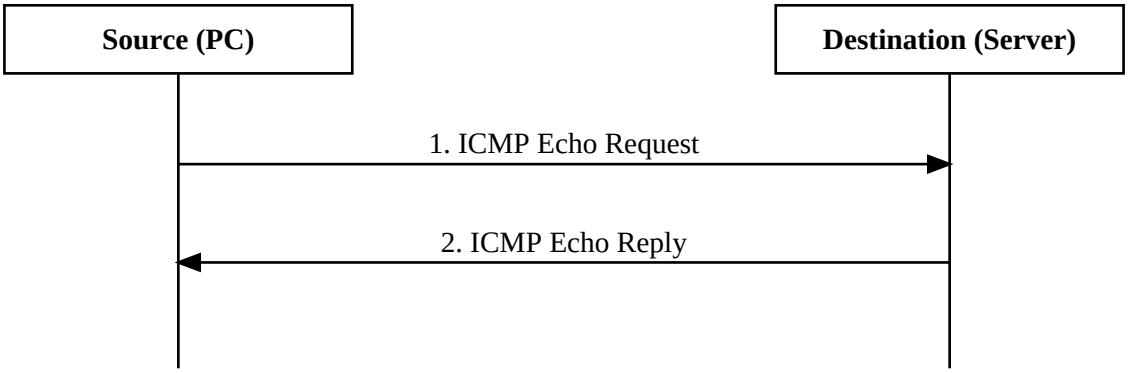


Fig 2: Working of ping command



OUTPUT:

Linux:

```
USER @USER -VirtualBox: ~
```

```
USER   @USER   -VirtualBox:~$ hostname
USER   -VirtualBox
```

```
USER @USER -VirtualBox: ~
```

```
USER   @USER   -VirtualBox:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.10  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::a00:27ff:fe4e:66a1  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:4e:66:a1  txqueuelen 1000  (Ethernet)
        RX packets 2543  bytes 2154321 (2.1 MB)
        TX packets 1987  bytes 312456 (312.4 KB)

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
```

USER @USER -VirtualBox: ~

```
USER   @USER   -VirtualBox:~$ ping -c 4 google.com
PING google.com (142.250.195.78) 56(84) bytes of data.
64 bytes from maa03s45-in-f14.1e100.net (142.250.195.78): icmp_seq=1 ttl=117 time=14.2 ms
64 bytes from maa03s45-in-f14.1e100.net (142.250.195.78): icmp_seq=2 ttl=117 time=14.8 ms
64 bytes from maa03s45-in-f14.1e100.net (142.250.195.78): icmp_seq=3 ttl=117 time=13.9 ms
64 bytes from maa03s45-in-f14.1e100.net (142.250.195.78): icmp_seq=4 ttl=117 time=15.2 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 13.904/14.525/15.230/0.512 ms
```

USER @USER -VirtualBox: ~

```
USER   @USER   -VirtualBox:~$ traceroute google.com
traceroute to google.com (142.250.195.78), 30 hops max, 60 byte packets
 1  _gateway (192.168.1.1)  1.102 ms  1.045 ms  0.987 ms
 2  10.20.0.1 (10.20.0.1)  4.321 ms  4.287 ms  4.410 ms
 3  172.16.5.9 (172.16.5.9)  8.765 ms  8.801 ms  8.730 ms
 4  72.14.215.85 (72.14.215.85)  12.543 ms  12.498 ms  12.610 ms
 5  maa03s45-in-f14.1e100.net (142.250.195.78)  14.230 ms  14.187 ms  14.301 ms
```

USER @USER -VirtualBox: ~

```
USER   @USER   -VirtualBox:~$ nslookup google.com
Server:         192.168.1.1
Address:        192.168.1.1#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.195.78
```

USER @USER -VirtualBox: ~

```
USER   @USER   -VirtualBox:~$ dig google.com +short
142.250.195.78
```

USER @USER -VirtualBox: ~

```
USER   @USER   -VirtualBox:~$ netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN
tcp6       0      0 :::80                   :::*                    LISTEN
udp        0      0 0.0.0.0:68              0.0.0.0:*
```

USER @USER -VirtualBox: ~

```
USER   @USER   -VirtualBox:~$ arp -a
_gateway (192.168.1.1) at c4:e9:84:2a:5b:10 [ether] on enp0s3
? (192.168.1.12) at 3c:52:82:7d:11:9e [ether] on enp0s3
```

USER @USER -VirtualBox: ~

```
USER   @USER   -VirtualBox:~$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         192.168.1.1     0.0.0.0         UG    100    0        0 enp0s3
192.168.1.0     0.0.0.0         255.255.255.0   U     100    0        0 enp0s3
```

Windows:

Command Prompt

```
C:\Users\USER   >hostname
DESKTOP-USER
```

Command Prompt

```
C:\Users\USER   >ipconfig /all

Windows IP Configuration

   Host Name . . . . . . . . . . . . : DESKTOP-USER
   Primary Dns Suffix  . . . . . . . :
   Node Type . . . . . . . . . . . . : Hybrid
   IP Routing Enabled. . . . . . . . : No

Ethernet adapter Ethernet:

   Physical Address. . . . . . . . . : 00-1A-2B-3C-4D-5E
   DHCP Enabled. . . . . . . . . . . : Yes
   IPv4 Address. . . . . . . . . . . : 192.168.1.25(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.1.1
   DHCP Server . . . . . . . . . . . : 192.168.1.1
   DNS Servers . . . . . . . . . . . : 8.8.8.8
```

Command Prompt

```
C:\Users\USER   >ping google.com

Pinging google.com [142.250.195.78] with 32 bytes of data:
Reply from 142.250.195.78: bytes=32 time=15ms TTL=117
Reply from 142.250.195.78: bytes=32 time=14ms TTL=117
Reply from 142.250.195.78: bytes=32 time=14ms TTL=117
Reply from 142.250.195.78: bytes=32 time=16ms TTL=117

Ping statistics for 142.250.195.78:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 14ms, Maximum = 16ms, Average = 14ms
```

Command Prompt

```
C:\Users\USER   >tracert google.com

Tracing route to google.com [142.250.195.78]
over a maximum of 30 hops:

  1    <1 ms    <1 ms    <1 ms  192.168.1.1
  2     4 ms     4 ms     4 ms  10.20.0.1
  3     9 ms     8 ms     9 ms  172.16.5.9
  4    13 ms    12 ms    12 ms  72.14.215.85
  5    14 ms    14 ms    15 ms  142.250.195.78

Trace complete.
```

Command Prompt

```
C:\Users\USER   >nslookup google.com
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:    google.com
Address:  142.250.195.78
```

Command Prompt

```
C:\Users\USER   >netstat -an

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING
  TCP    0.0.0.0:445            0.0.0.0:0              LISTENING
  TCP    192.168.1.25:50234     142.250.195.78:443     ESTABLISHED
  UDP    0.0.0.0:5353           *:*
```

Command Prompt

```
C:\Users\USER   >arp -a

Interface: 192.168.1.25 --- 0x8
  Internet Address      Physical Address      Type
  192.168.1.1           c4-e9-84-2a-5b-10     dynamic
  192.168.1.255         ff-ff-ff-ff-ff-ff     static
```

Command Prompt

```
C:\Users\USER   >route print

IPv4 Route Table
===========================================================================
Active Routes:
Network Destination        Netmask          Gateway       Interface  Metric
          0.0.0.0          0.0.0.0      192.168.1.1    192.168.1.25     25
        127.0.0.0        255.0.0.0         On-link         127.0.0.1    331
      192.168.1.0    255.255.255.0         On-link      192.168.1.25    281
```

Command Prompt

```
C:\Users\USER   >getmac

Physical Address    Transport Name
=================== ==========================================================
00-1A-2B-3C-4D-5E   \Device\Tcpip_{8F3A21B0-5C6D-4E7F-9A1B-2C3D4E5F6A7B}
```

RESULT:

Thus the various network commands used in Linux and Windows were studied and executed successfully.

Exp No: 2	Study of Network cables - straight-through and crossover cable

AIM:

To study the different types of network cables and to make a straight-through cable and a cross-wired (crossover) cable using a crimping tool.

APPARATUS REQUIRED:

- Cat5e / Cat6 UTP cable (about 1 metre for each cable)
- RJ45 connectors (4 numbers)
- Crimping tool with built-in cutter and stripper
- LAN cable tester (main unit and remote unit)

ALGORITHM / PROCEDURE:

Part A - Study of network cables

1. Observe the sample twisted pair, coaxial and fibre optic cables available in the lab.
2. Note the outer jacket marking (for example Cat5e, 24 AWG) and the number of pairs.
3. Write down the use, speed and maximum length of each cable type in the table given below.

Part B - Making the cables

1. Cut the UTP cable to the required length (about 1 m) using the cutter.
2. Remove about 2.5 cm of the outer jacket from both ends with the stripper. Take care not to cut the inner wires.
3. Untwist the four pairs and straighten the eight wires with fingers.
4. For the **straight-through cable**: arrange the wires in `T568B` order on both ends.
5. For the **crossover cable**: arrange `T568B` order on one end and `T568A` order on the other end.
6. Cut the wire tips even so that about 1.3 cm of the wires is left outside the jacket.
7. Hold the RJ45 connector with the clip facing down, push the wires inside till they touch the end of the connector. The jacket should also go inside a little.
8. Put the connector in the crimping tool and press firmly so that the pins cut into the wires.
9. Repeat steps 2 to 8 for the other end of the cable.
10. Insert both ends into the LAN tester and check the LED sequence.

Types of network cables

Cable	Connector	Max. length	Speed	Used for
Twisted pair (UTP / STP)	RJ45	100 m	10 Mbps to 10 Gbps	LAN, connecting PCs to switch
Coaxial (RG-6, RG-59)	BNC / F-type	185 m to 500 m	10 Mbps (old Ethernet)	Cable TV, CCTV, old LANs
Fibre optic (single / multi mode)	SC / LC / ST	2 km (multi) up to 40 km+ (single)	1 Gbps to 100 Gbps	Backbone, long distance links

UTP category	Max. speed	Bandwidth
Cat5	100 Mbps	100 MHz

UTP category	Max. speed	Bandwidth
Cat5e	1 Gbps	100 MHz
Cat6	1 Gbps (10 Gbps up to 55 m)	250 MHz
Cat6a	10 Gbps	500 MHz

Pin	1	2	3	4	5	6	7	8
T568A	W/G	G	W/O	B	W/B	O	W/Br	Br
T568B	W/O	O	W/G	B	W/B	G	W/Br	Br

W = white stripe, G = green, O = orange, B = blue, Br = brown

Cable	Wiring	Used to connect
Straight-through	T568B to T568B	PC to Switch, PC to Hub, Router to Switch (unlike devices)
Crossover	T568B to T568A	PC to PC, Switch to Switch, Hub to Hub, PC to Router (like devices)

DIAGRAM:

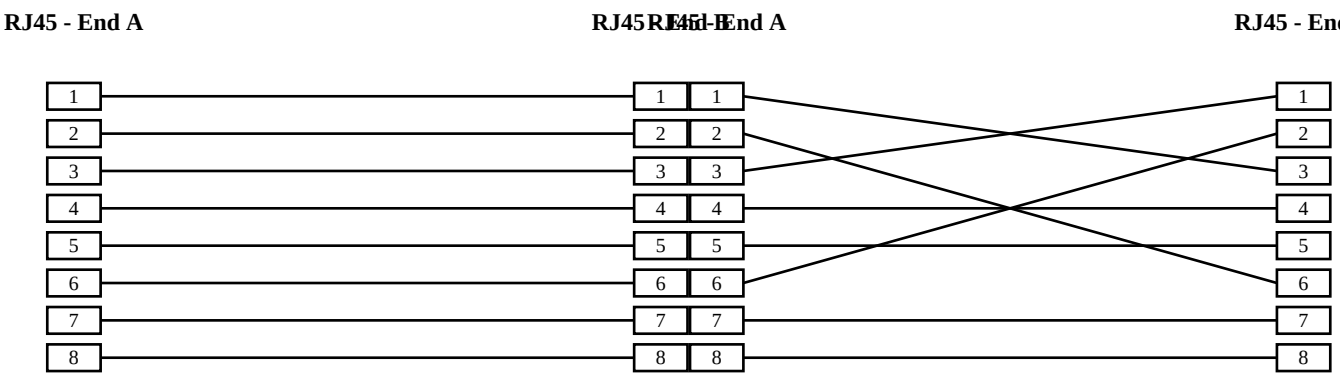
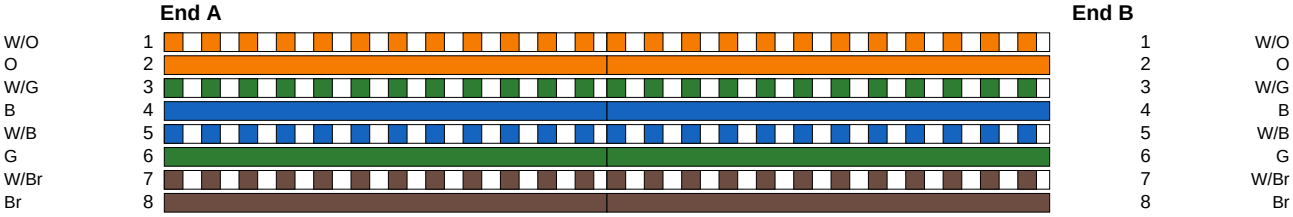


Fig 1: Straight-through cable (pin 1 to 1, 2 to 2 ...)

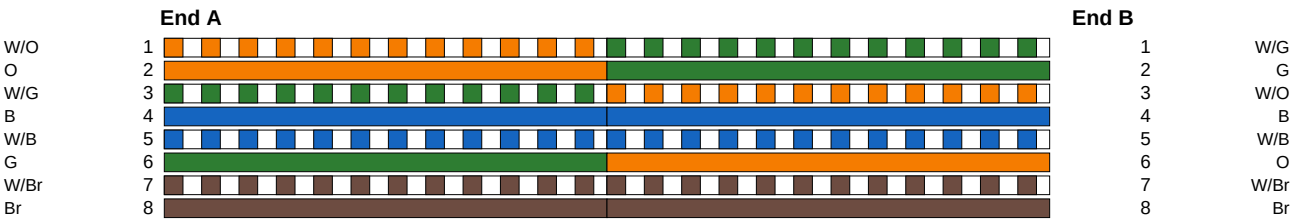
Fig 2: Crossover cable (1 to 3, 2 to 6, 3 to 1, 6 to 2)

OUTPUT:

Colour order of the wires at both ends

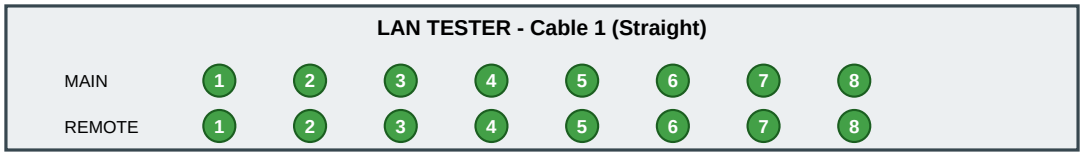


Cable 1: Straight-through cable (T568B - T568B)

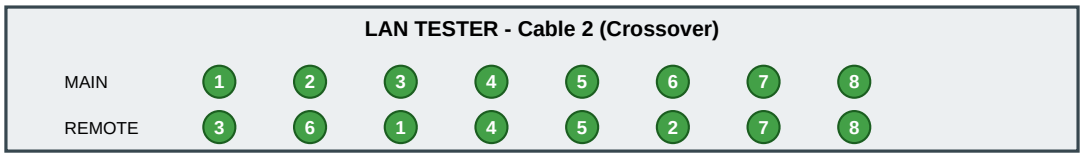


Cable 2: Crossover cable (T568B - T568A)

LAN cable tester result



LEDs glow in the order 1-2-3-4-5-6-7-8 : cable is OK



LEDs glow in the order 3-6-1-4-5-2-7-8 : crossover cable is OK

Cable	Main unit LED	Remote unit LED	Status
Straight-through	1 2 3 4 5 6 7 8	1 2 3 4 5 6 7 8	Working
Crossover	1 2 3 4 5 6 7 8	3 6 1 4 5 2 7 8	Working

RESULT:

Thus the different types of network cables were studied. A straight-through cable and a crossover cable were made using the crimping tool and both were tested successfully with the LAN tester.

Exp No: 3	*Experiments on CISCO PACKET TRACER - Hub and Switch*

AIM:

To understand the environment of Cisco Packet Tracer, to design simple networks using a hub and a switch with 4 to 6 hosts, and to analyse the behaviour of these devices by tracing the communication in simulation mode.

ALGORITHM / PROCEDURE:

1. Open Cisco Packet Tracer. Study the workspace: device selection box (bottom left), logical workspace (centre), Realtime / Simulation switch (bottom right).
2. Select ***End Devices*** and drag 5 PCs and a Hub on to the workspace.
3. Select ***Connections*** and use the ***Automatically Choose Connection Type*** (copper straight-through) to connect each PC to the hub.
4. Click a PC, go to ***Desktop > IP Configuration*** and set the IP address 192.168.1.1 to 192.168.1.5 with subnet mask 255.255.255.0.
5. Open ***Desktop > Command Prompt*** of PC0 and ping the other PCs to check the connectivity.
6. Switch to ***Simulation*** mode, click ***Add Simple PDU*** (envelope) and click PC0 then PC3. Press ***Capture / Forward*** and watch how the packet moves.
7. Repeat steps 2 to 6 by replacing the hub with a ***Switch*** (Experiment 2).
8. Build a network that has both a hub and a switch (Experiment 3), connect hub and switch with a crossover cable, send a PDU from a hub PC to a switch PC and compare the behaviour.
9. Observe the event list and the MAC address table of the switch show mac-address-table.

DIAGRAM:

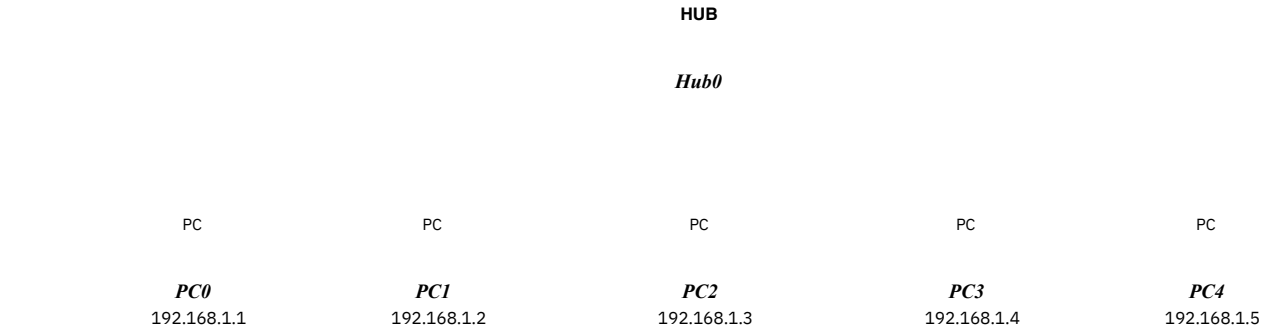
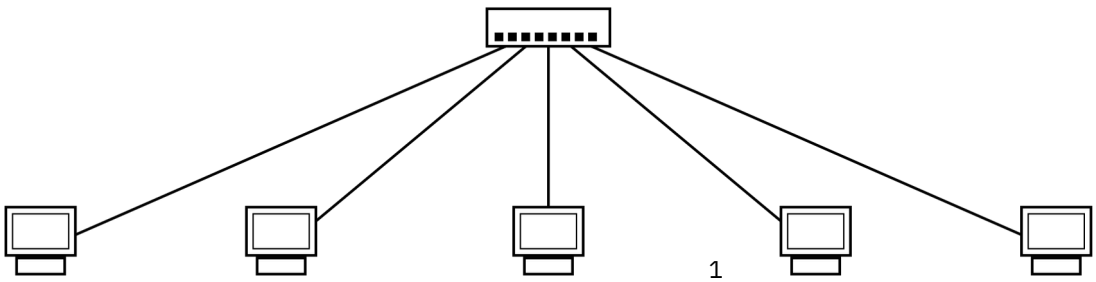


Fig 1: Network using only a HUB (5 hosts)



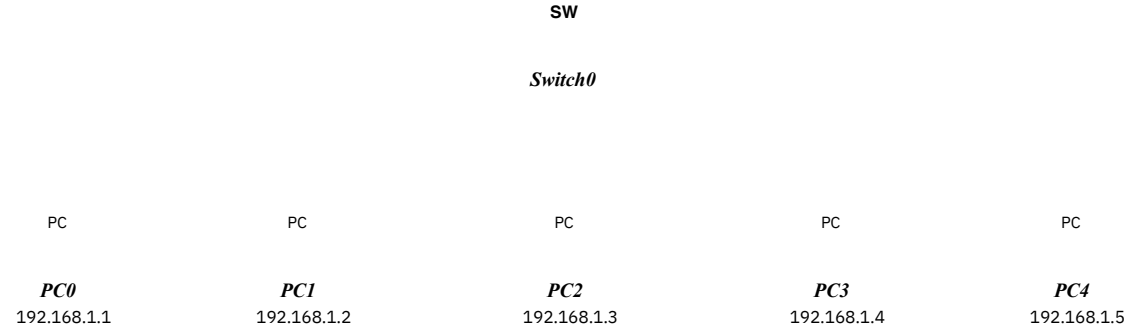


Fig 2: Network using only a SWITCH (5 hosts)

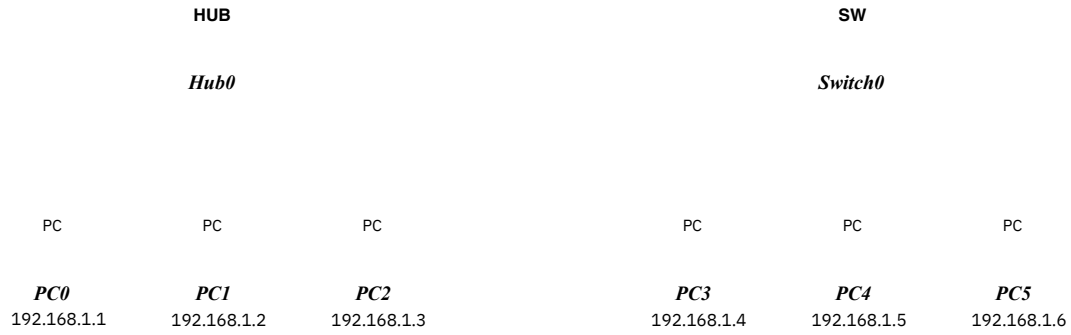


Fig 3: Network using both HUB and SWITCH (6 hosts)

Network topology implemented in our college

The college network follows an ***extended star (hierarchical) topology***. The computers of each lab are connected to a lab switch with straight-through UTP cables (star). All lab switches are connected to the core switch kept in the server room and the core switch goes to the router / firewall and then to the ISP.

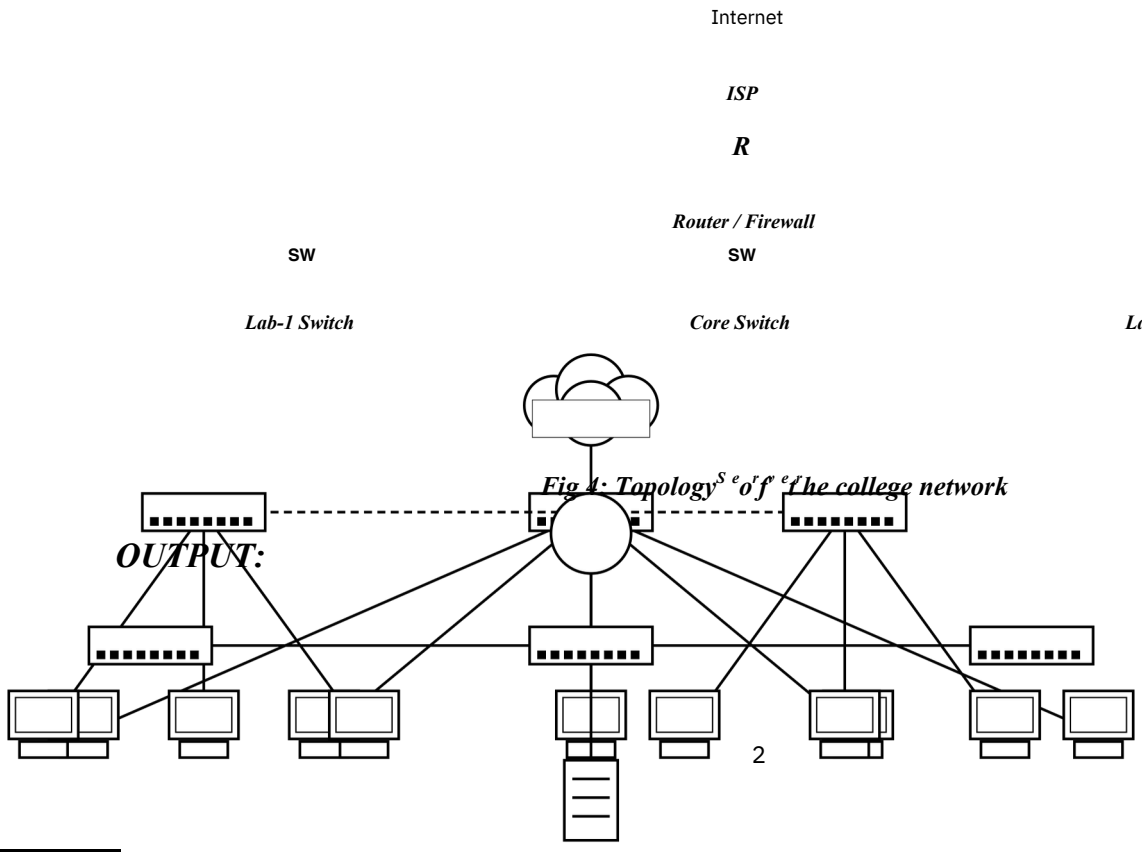
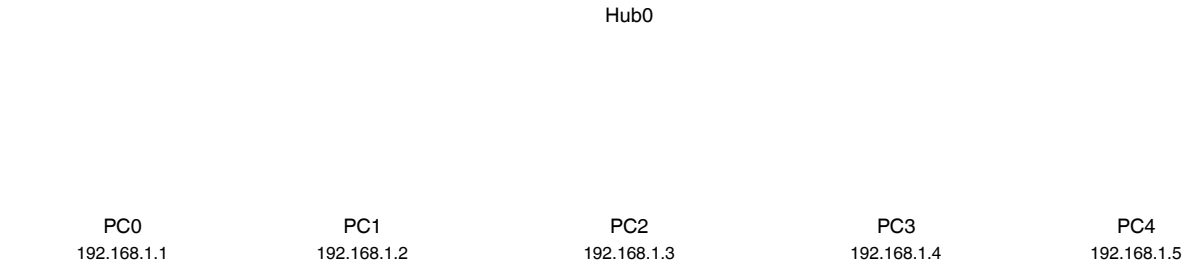


Fig 4: Topology of the college network

OUTPUT:



Experiment 1 : hub network in Packet Tracer (all link lights green)

PC0 - Command Prompt

```
C:\>

Pinging 192.168.1.4 with 32 bytes of data:

Request timed out.
Reply from 192.168.1.4: bytes=32 time=6ms TTL=128
Reply from 192.168.1.4: bytes=32 time<1ms TTL=128
Reply from 192.168.1.4: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.4:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 6ms, Average = 2ms

C:\>

Pinging 192.168.1.4 with 32 bytes of data:

Reply from 192.168.1.4: bytes=32 time<1ms TTL=128
Reply from 192.168.1.4: bytes=32 time<1ms TTL=128
Reply from 192.168.1.4: bytes=32 time<1ms TTL=128
Reply from 192.168.1.4: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.4:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

The first ping loses one packet because the ARP request has to be resolved first. The second ping shows 0% loss.

Simulation mode - Hub (PC0 to PC3)

Fire	Last Status	Source	Destination	Type	Color	Time(sec	Periodic	Num
	Successful	PC0	Hub0	ARP		) 0.000	N	0
	Successful	Hub0	PC1	ARP		0.001	N	1
	Successful	Hub0	PC2	ARP		0.001	N	2
	Successful	Hub0	PC3	ARP		0.001	N	3
	Successful	Hub0	PC4	ARP		0.001	N	4
	Failed	PC1	--	ICMP		0.002	N	5
	Successful	PC0	Hub0	ICMP		0.003	N	6

The hub repeats every frame on all ports. PC1, PC2 and PC4 receive the frame and drop it because it is not meant for them.

Switch0

PC0	PC1	PC2	PC3	PC4
192.168.1.1	192.168.1.2	192.168.1.3	192.168.1.4	192.168.1.5

Experiment 2 : switch network in Packet Tracer

Switch0 - CLI

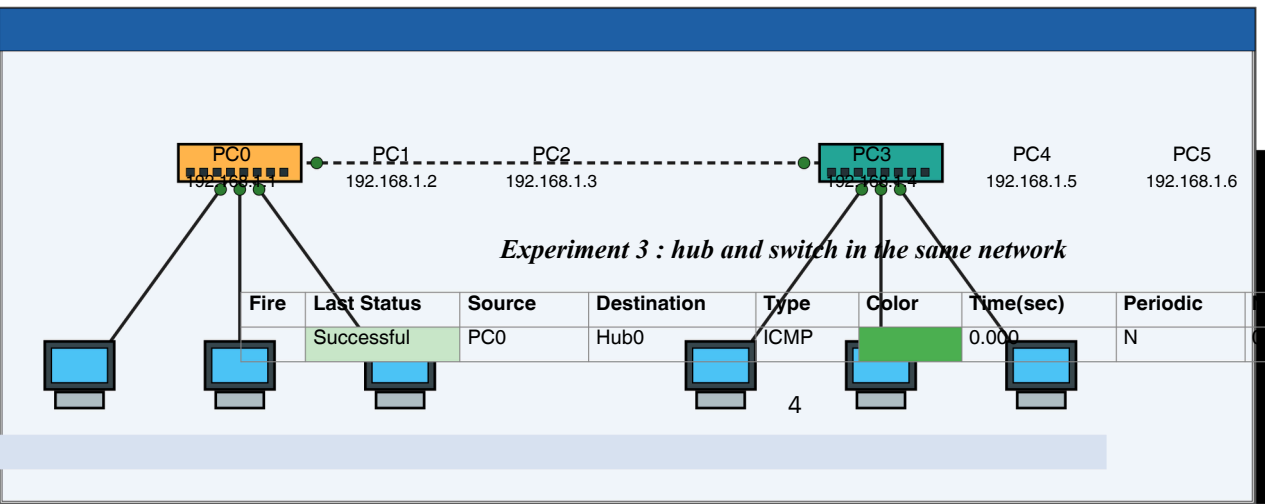
```
enable
show mac-address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address --    Type ---    Ports
----    ---------         -----       -----
   1    0001.4382.a3c1    DYNAMIC     Fa0/1
   1    0002.1779.b4d2    DYNAMIC     Fa0/4
   1    0060.2fa1.0c55    DYNAMIC     Fa0/2
```

Fire	Last Status	Source	Destination	Type	Color	Time(sec	Periodic	Num
	Successful	PC0	Switch0	ARP		) 0.000	N	0
	Successful	Switch0	PC1	ARP		0.001	N	1
	Successful	Switch0	PC2	ARP		0.001	N	2
	Successful	Switch0	PC3	ARP		0.001	N	3
	Successful	PC3	Switch0	ARP		0.002	N	4
	Successful	PC0	Switch0	ICMP		0.003	N	5
	Successful	Switch0	PC3	ICMP		0.004	N	6

Only the first ARP broadcast is flooded. After the switch has learnt the MAC addresses, the ICMP frame goes only to the port of PC3.

Hub0 Switch0



Experiment 3 : hub and switch in the same network

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	n
	Successful	PC0	Hub0	ICMP		0.000	N	

	Successful	Hub0	PC1	ICMP		0.001	N	1
	Successful	Hub0	PC2	ICMP		0.001	N	2
	Successful	Hub0	Switch0	ICMP		0.001	N	3
	Successful	Switch0	PC3	ICMP		0.002	N	4
	Successful	PC3	Switch0	ICMP		0.003	N	5
	Successful	Switch0	Hub0	ICMP		0.004	N	6

Point	***Hub***	***Switch***
OSI layer	Physical layer (Layer 1)	Data link layer (Layer 2)
Forwarding	Repeats the signal to all ports	Forwards the frame only to the destination port
Address table	None	MAC address table (learning)
Collision domain	One for all the ports	One for each port
Bandwidth	Shared among all hosts	Dedicated to each port
Duplex	Half duplex	Full duplex
Security / traffic	Anyone can capture the traffic	Traffic is limited to the two hosts

RESULT:

Thus the Cisco Packet Tracer environment was studied. Networks using only a hub, only a switch, and both a hub and a switch were designed and the packet flow was traced in simulation mode. The hub broadcasts every frame to all ports whereas the switch learns MAC addresses and forwards the frame only to the destination port.

Exp No: 4	*Setup and configure a LAN using a Switch and Ethernet cables*

AIM:

To set up and configure a LAN using a switch and Ethernet cables, assign IP addresses to the hosts, check the connectivity using the ping command and share files and folders across the machines of the LAN.

ALGORITHM / PROCEDURE:

1. Connect 4 host machines to the switch using straight-through UTP cables (RJ45). Check that the link LEDs of the switch glow.
2. Assign a static IP address to each host: ***Windows***, Control Panel > Network and Sharing Centre > Change adapter settings > Ethernet > Properties > IPv4 (or run ncpa.cpl).
3. Use the addresses 192.168.10.11 to 192.168.10.14 with subnet mask 255.255.255.0. Leave the default gateway empty because there is only one network.
4. Open the Command Prompt and verify the settings with ipconfig.
5. Check the connectivity between the machines with ping, for example from PC1 ping `192.168.10.12`.
6. On PC1 create a folder LabShare, right click > Properties > Sharing > Advanced Sharing > tick ***Share this folder*** and give ***Read/Write*** permission to ***Everyone***.
7. On PC2 open ***File Explorer*** and type \\192.168.10.11 in the address bar. The files of PC1 are displayed.
8. The folder can also be mapped as a drive with net use Z: \\192.168.10.11\LabShare and the shared files can be copied between the machines.
9. Note down the output of each command.

Host	*IP address*	*Subnet mask*	*Switch port*
PC1	192.168.10.11	255.255.255.0	Fa0/1
PC2			Fa0/2
PC3	192.168.10.12	255.255.255.0	Fa0/3
PC4	192.168.10.13	255.255.255.0	Fa0/4
	192.168.10.14	255.255.255.0	

DIAGRAM:

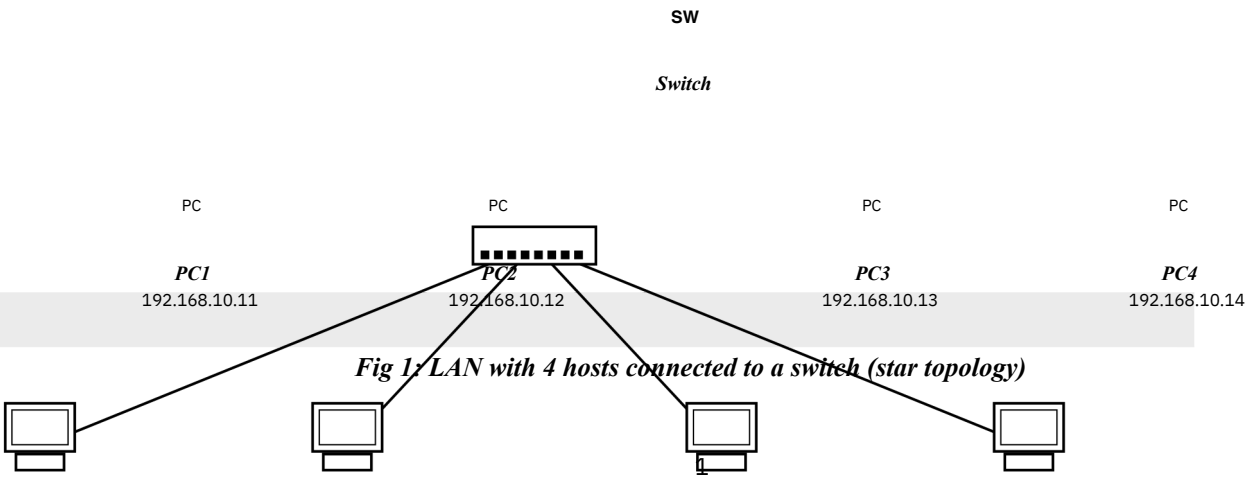


Fig 1: LAN with 4 hosts connected to a switch (star topology)

OUTPUT:

Command Prompt - PC1

```
C:\Users\UusSeErR>

Windows IP Configuration

Ethernet adapter Ethernet:

   Connection-specific DNS Suffix . :
   Link-local IPv6 Address . . . . . : fe80::6c1f:3a2e:91b4:d7c2%8
   IPv4 Address. . . . . . . . . . . : 192.168.10.11
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :
```

Command Prompt - PC1

```
C:\Users\UusSeErR>

Pinging 192.168.10.12 with 32 bytes of data:
Reply from 192.168.10.12: bytes=32 time=1ms TTL=128
Reply from 192.168.10.12: bytes=32 time<1ms TTL=128
Reply from 192.168.10.12: bytes=32 time<1ms TTL=128
Reply from 192.168.10.12: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.10.12:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\Users\UusSeErR>

Pinging 192.168.10.13 with 32 bytes of data:
Reply from 192.168.10.13: bytes=32 time<1ms TTL=128
Reply from 192.168.10.13: bytes=32 time<1ms TTL=128
Reply from 192.168.10.13: bytes=32 time=1ms TTL=128
Reply from 192.168.10.13: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.10.13:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\Users\UusSeErR>ping 192.168.10.14

Pinging 192.168.10.14 with 32 bytes of data:
Reply from 192.168.10.14: bytes=32 time<1ms TTL=128
Reply from 192.168.10.14: bytes=32 time<1ms TTL=128
Reply from 192.168.10.14: bytes=32 time<1ms TTL=128
Reply from 192.168.10.14: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.10.14:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
```

Command Prompt - PC2

```
C:\Users\UusSeErR >net view \\192.168.10.11
Shared resources at \\192.168.10.11

Share name   Type   Used as   Comment
-------------------------------------------------
LabShare     Disk
The command completed successfully.

C:\Users\UusSeErR >net use Z: \\192.168.10.11\LabShare
The command completed successfully.

C:\Users\UusSeErR >dir Z:
 Volume in drive Z is LabShare
Directory of Z:\

07-10-2026  08:42 PM    <DIR>          .
```

Command Prompt - PC2 (contd.)

```
07-10-2026  08:42PM     <DIR>          ..
07-10-2026  08:40PM             14,336 CN_Lab_Notes.docx
07-10-2026  08:41PM            212,480 Exp4_Topology.png
07-10-2026  08:42PM              1,024 hosts_list.txt
               3File(s)        227,840 bytes
               2Dir(s)  48,213,327,872 bytes free

C:\Users\UusSeErR >
        1 file(s) copied.
```

RESULT:

Thus a LAN was set up with 4 hosts connected to a switch using Ethernet cables, the IP addresses were assigned, the connectivity was verified using the ping command and files were shared and accessed across the machines of the LAN.

Exp No: 5	*Experiments on Packet capture tool: Wireshark*

AIM:

To understand the features of Wireshark as a packet capture tool and to understand the encapsulation of information at the various layers of a protocol stack.

ALGORITHM / PROCEDURE:

1. Install and open Wireshark. The home screen lists all the network interfaces with a small activity graph.
2. Double click the active interface (Ethernet / Wi-Fi) to start the capture.
3. Generate some traffic: open a website in the browser, run ping google.com and nslookup example.com in the Command Prompt.
4. Stop the capture using the red square button.
5. Apply display filters in the filter bar, for example icmp, dns, tcp.port == 80, ip.addr == `192.168.1.25`.
6. Select one packet and expand its layers in the ***Packet Details*** pane. Study the Frame, Ethernet II, Internet Protocol, TCP / ICMP headers.
7. Select any header field and see the corresponding bytes highlighted in the ***Packet Bytes*** pane.
8. Use ***Analyze > Follow > TCP Stream*** to follow the full TCP conversation and note the three-way handshake (SYN, SYN-ACK, ACK).
9. Save the capture using ***File > Save As (.pcapng)***.

Features of Wireshark

- Captures live packets from Ethernet, Wi-Fi, loopback and other interfaces and also reads saved .pcap files.
- Capture filters (before capture) and display filters (after capture).
- Colouring rules to identify the protocols easily (TCP, UDP, ICMP, ARP, HTTP ...).
- Shows all the protocol headers layer by layer with the raw bytes.
- Statistics tools like Conversations, Protocol Hierarchy and I/O Graph.

Layer	*Header added*	*Protocol example*	*PDU name*	
Application	Application header	HTTP, DNS	Data / Message	
Transport	Port numbers, sequence no. (TCP 20 B, UDP 8 B)	TCP, UDP	Segment / Datagram	
Network	Source and destination IP address (20 B)	IP, ICMP	Packet	
Data link	Source and destination MAC (14 B) + FCS (4 B)	Ethernet	Frame	
Physical	None - only signals	UTP, Wi-Fi	Bits	

DIAGRAM:

Application layer	DATA	***Data***
Transport layer	TCP/ICMP header + DATA	***Segment***
Network layer	IP header | TCP header | DATA	***Packet***
Data link layer	Eth header | IP | TCP | DATA | FCS	***Frame***
Physical layer	0 1 0 1 1 0 0 1 0 1 1 1 ...	***Bits***

Fig 1: Encapsulation of data as it moves down the protocol stack

OUTPUT:

No.	Time	Source	Destination	Protocol	Len	Info
1	0.000000	192.168.1.25	192.168.1.1	DNS	70	Standard query 0x3a1c A example.com
2	0.012431	192.168.1.1	192.168.1.25	DNS	86	Standard query response 0x3a1c A example.com A 93.184.216.34
3	0.013102	192.168.1.25	93.184.216.34	TCP	66	51322 > 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
4	0.041875	93.184.216.34	192.168.1.25	TCP	66	80 > 51322 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
5	0.041932	192.168.1.25	93.184.216.34	TCP	54	51322 > 80 [ACK] Seq=1 Ack=1 Win=131328 Len=0
6	0.042210	192.168.1.25	93.184.216.34	HTTP	142	GET / HTTP/1.1
7	0.071655	93.184.216.34	192.168.1.25	TCP	54	80 > 51322 [ACK] Seq=1 Ack=89 Win=65535 Len=0
8	0.093340	93.184.216.34	192.168.1.25	HTTP	1294	HTTP/1.1 200 OK (text/html)
9	1.402118	192.168.1.25	142.250.195.78	ICMP	74	Echo (ping) request id=0x0001, seq=21/5376, ttl=128
10	1.416875	142.250.195.78	192.168.1.25	ICMP	74	
11	2.118004	c4:e9:84:2a:5b:10	Broadcast	ARP	60	Echo (ping) reply id=0x0001, seq=21/5376, ttl=117
						Who has192.168.1.25?Tell192.168.1.1

Packet List pane - captured packets (colours show the protocol)

Packet Details - Frame 9 (ICMP Echo request)

```
v Frame 9: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface \Device\NPF_{8F3A21B0}
, id 0
    Arrival Time: Oct  7, 2026 20:57:38.402118000 India Standard Time
    Frame Length: 74 bytes (592 bits)
    Protocols in frame: eth:ethertype:ip:icmp:data
v Ethernet II, Src: 00:1a:2b:3c:4d:5e, Dst: c4:e9:84:2a:5b:10
    Destination: c4:e9:84:2a:5b:10
    Source: 00:1a:2b:3c:4d:5e
    Type: IPv4 (0x0800)
v Internet Protocol Version 4, Src: 192.168.1.25, Dst: 142.250.195.78
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Total Length: 60
    Identification: 0x7a31 (31281)
    Time to Live: 128
    Protocol: ICMP (1)
    Source Address: 192.168.1.25
    Destination Address: 142.250.195.78
v Internet Control Message Protocol
    Type: 8 (Echo (ping) request)
    Code: 0
    Checksum: 0x4d5a [correct]
    Identifier (BE): 1 (0x0001)
    Sequence Number (BE): 21 (0x0015)
    Data (32 bytes)
```

Packet Bytes - Frame 9

```
0000  c4 e9 84 2a 5b 10 00 1a 2b 3c4d5e08004500 00 3c  ...*[...+<M^..E.
0010  7a 31 00 00 80 01 00 00c0a801198efa c3 4e 08 00  .<z1.............
0020  4d 5a 00 01 00 15616263646566 67 68 69 6a 6b 6c  .N..MZ....abcdef
0030  6d 6e 6f 70717273747576 77 61 62 63 64 65 66 67  ghijklmnopqrstuv
0040  68 69                                           wabcdefghi
```

Layer	*Bytes in the frame*	*Observed value*
Ethernet II	00 - 13 (14 bytes)	c4:e9:84:2a:5b:10 <- 00:1a:2b:3c:4d:5e, type 0x0800
IPv4	14 - 33 (20 bytes)	192.168.1.25 -> 142.250.195.78, TTL 128, protocol 1 (ICMP)
ICMP	34 - 41 (8 bytes)	Type 8, Code 0, id 1, seq 21
Data	42 - 73 (32 bytes)	abcdefghijklmnopqrstuvwabcdefghi

RESULT:

Thus the features of Wireshark were studied. Packets were captured and the encapsulation of data at the Data link (Ethernet), Network (IP), Transport (TCP) and ICMP layers was observed in the packet details.

Exp No: 6	*Error Correction at Data Link Layer using Hamming Code*

AIM:

To write a program that detects and corrects a single-bit error in a data stream using the Hamming (7,4) code, and to verify the error correction feature with a test run.

ALGORITHM / PROCEDURE:

1. Take the 4 data bits d1 d2 d3 d4 as input from the user.
2. Place the bits in a 7-bit codeword. Positions 1, 2 and 4 (powers of 2) are kept for the parity bits p1, p2, p3 and the rest hold the data bits.
3. Calculate the even parity bits: p1 = d1 ^ d2 ^ d4, p2 = d1 ^ d3 ^ d4, p3 = d2 ^ d3 ^ d4.
4. Send the 7-bit codeword. Flip one bit (by user input) to act as a transmission error.
5. At the receiver, recompute the three parity checks s1, s2 and s3 using the received bits.
6. The binary number s3 s2 s1 is the syndrome. If it is 000 there is no error, otherwise it is the position of the wrong bit.
7. Flip the bit at that position to correct it and extract the original data bits.
8. Compare the recovered data with the original data and display the result.

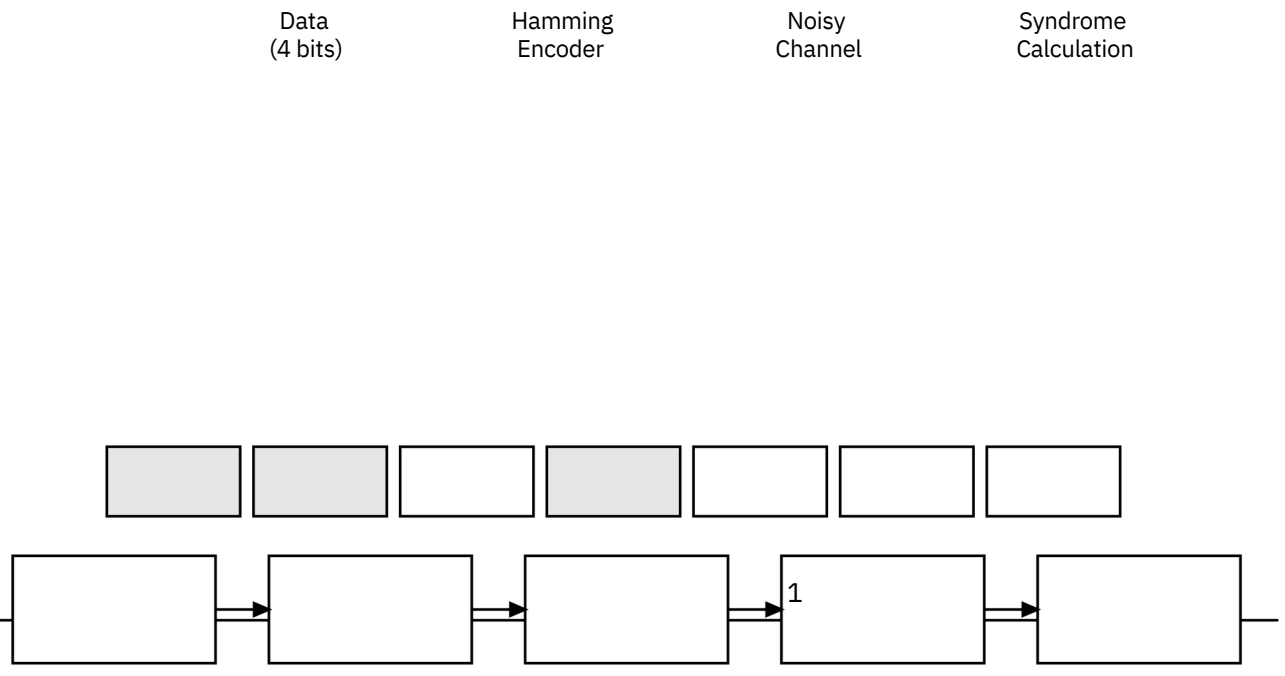
Fig 1: Hamming (7,4) codeword layout

Pos	1	2	3	4	5	6	7
Bit	***p1***	***p2***	***d1***	***p3***	***d2***	***d3***	***d4***

p1 checks positions 1,3,5,7 p2 checks 2,3,6,7 p3 checks 4,5,6,7

Shaded boxes = parity bits (positions 1, 2, 4). Unshaded = data bits.

Fig 2: Block diagram of error detection and correction



PROGRAM:

```
# Hamming (7,4) error detection and correction
def encode(d):                          # d = [d1, d2, d3, d4]
    d1, d2, d3, d4 = d
    p1 = d1 ^ d2 ^ d4
    p2 = d1 ^ d3 ^ d4
    p3 = d2 ^ d3 ^ d4
    return [p1, p2, d1, p3, d2, d3, d4]    # positions 1..7

def decode(c):
    s1 = c[0] ^ c[2] ^ c[4] ^ c[6]
    s2 = c[1] ^ c[2] ^ c[5] ^ c[6]
    s3 = c[3] ^ c[4] ^ c[5] ^ c[6]
    pos = s3 * 4 + s2 * 2 + s1            # 0 means no error
    if pos != 0:
        c[pos - 1] ^= 1                   # correct the bit
    return pos, [c[2], c[4], c[5], c[6]]

data = [int(b) for b in input("Enter 4 data bits: ")]
code = encode(data)
print("Data bits         :", *data)
 print("Hamming codeword :", *code)

recv = code[:]
p = int(input("Enter bit position (1-7) to corrupt, 0 for none: "))
if p:
    recv[p - 1] ^= 1
print("Received word     :", *recv)

pos, corrected = decode(recv)
if pos == 0:
    print("No error detected.")
else:
    print("Error detected at position", pos)
    print("Corrected word    :", *recv)
print("Recovered  data  :", *corrected)
print("Verification      :", "SUCCESS" if corrected == data else "FAILED")
```

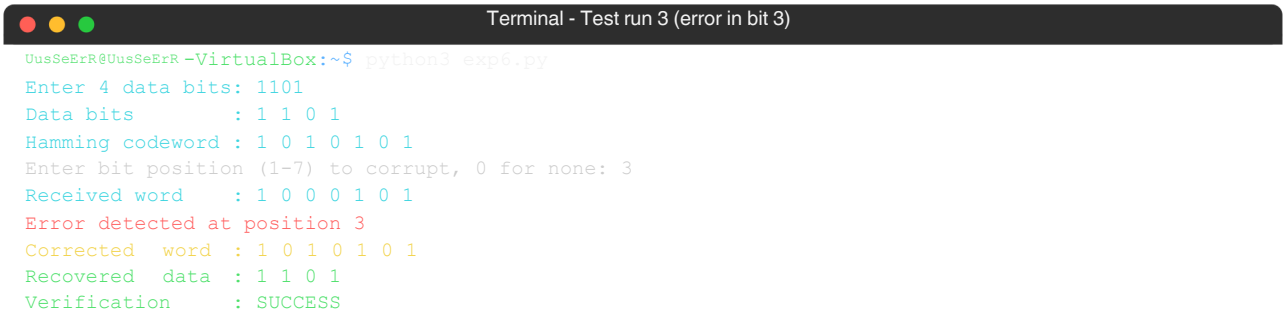
OUTPUT:

Terminal - Test run 1 (error in bit 6)

```
UusSeErR@UusSeErR-VirtualBox:~$ python3 exp6.py
Enter 4 data bits: 1011
Data bits         : 1 0 1 1
Hamming codeword : 0 1 1 0 0 1 1
Enter bit position (1-7) to corrupt, 0 for none: 6
Received word     : 0 1 1 0 0 0 1
Error detected at position 6
Corrected  word  : 0 1 1 0 0 1 1
Recovered  data  : 1 0 1 1
Verification      : SUCCESS
```

Terminal - Test run 2 (no error)

```
UusSeErR@UusSeErR-VirtualBox:~$ python3 exp6.py
Enter 4 data bits: 0110
Data bits         : 0 1 1 0
Hamming codeword : 1 1 0 0 1 1 0
Enter bit position (1-7) to corrupt, 0 for none: 0
Received word     : 1 1 0 0 1 1 0
No error detected.
Recovered data    : 0 1 1 0
Verification      : SUCCESS
```



RESULT:

Thus the program for error detection and correction using Hamming code was written and executed successfully. A single-bit error was found at the correct position and corrected in every test run.

Exp No: 7	*Flow Control at Data Link Layer using Sliding Window Protocol*

AIM:

To write a program that implements flow control at the data link layer using the Sliding Window Protocol (Go-Back-N) and to simulate the flow of frames from one node to another.

ALGORITHM / PROCEDURE:

1. Read the total number of frames and the window size N. The sender may send up to N frames without waiting for an acknowledgement.
2. Keep two pointers at the sender: base (oldest unacknowledged frame) and nxt (next frame to send).
3. While nxt < base + N, send frame number nxt and increase nxt.
4. The receiver accepts frames in order. For each correct frame it sends an acknowledgement ACK (frame number + 1).
5. When the sender receives an ACK, the window slides forward by one position (base = base
6. + 1). If a frame is lost the receiver discards the later frames, the sender times out and sends again from the lost frame (Go-Back-N).
7. Repeat until all the frames are acknowledged and display the sequence of events.

Fig 1: Sender window over the frame sequence

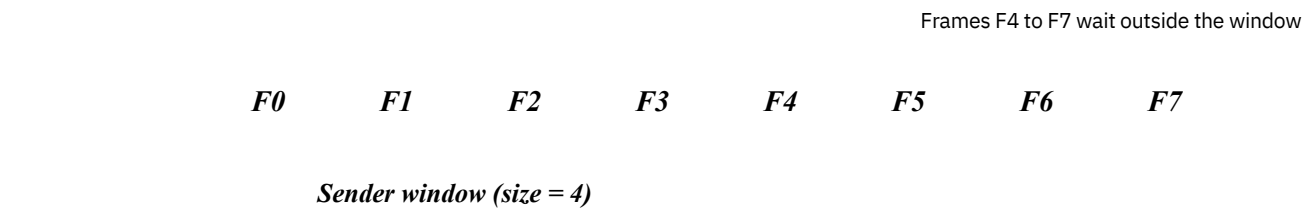
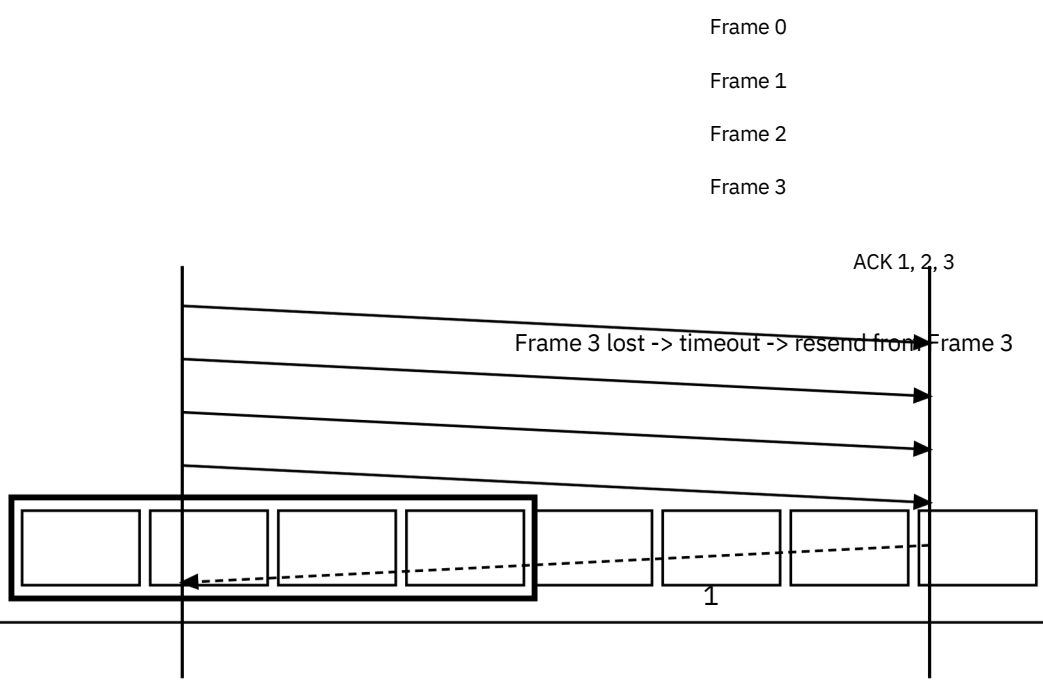


Fig 2: Frame and ACK exchange between sender and receiver



PROGRAM:

```
# Sliding window protocol (Go-Back-N) simulation
TOTAL, WINDOW = 8, 4
LOST = {3}                          # frame 3 is lost the first time it is sent

base = nxt = 0
lost_once = set(LOST)
print(f"Total frames = {TOTAL}, Window size = {WINDOW}\n")
while base < TOTAL:
    while nxt < base + WINDOW and nxt < TOTAL:
        print(f"Sender   : Frame {nxt} sent   window=[{base}..{min(base+WINDOW,TOTAL)-1}]")
        nxt += 1
    expected = base
    for f in range(base, nxt):
    if f in lost_once:
            print(f"Network : Frame {f} LOST")
            lost_once.discard(f)
            print(f"Sender   : Timeout for frame {f}, resending from frame {f}")
            nxt = f
            break
        print(f"Receiver : Frame {f} received, ACK {f + 1} sent")
        print(f"Sender   : ACK {f + 1} received, window slides")
        base = f + 1
print("\nAll frames transmitted successfully.")
```

OUTPUT:

Terminal - Sliding window simulation

```
UusSeErR@UusSeErR-VirtualBox:~$ python3 exp7.py
Total frames = 8, Window size = 4

Sender   : Frame 0 sent   window=[0..3]
Sender   : Frame 1 sent   window=[0..3]
Sender   : Frame 2 sent   window=[0..3]
Sender   : Frame 3 sent   window=[0..3]
Receiver : Frame 0 received, ACK 1 sent
Sender   : ACK 1 received, window slides
Receiver : Frame 1 received, ACK 2 sent
Sender   : ACK 2 received, window slides
Receiver : Frame 2 received, ACK 3 sent
Sender   : ACK 3 received, window slides
Network  : Frame 3 LOST
Sender   : Timeout for frame 3, resending from frame 3
Sender   : Frame 3 sent   window=[3..6]
Sender   : Frame 4 sent   window=[3..6]
Sender   : Frame 5 sent   window=[3..6]
Sender   : Frame 6 sent   window=[3..6]
Receiver : Frame 3 received, ACK 4 sent
Sender   : ACK 4 received, window slides
Receiver : Frame 4 received, ACK 5 sent
Sender   : ACK 5 received, window slides
Receiver : Frame 5 received, ACK 6 sent
Sender   : ACK 6 received, window slides
Receiver : Frame 6 received, ACK 7 sent
Sender   : ACK 7 received, window slides
Sender   : Frame 7 sent   window=[7..7]
Receiver : Frame 7 received, ACK 8 sent
Sender   : ACK 8 received, window slides

All frames transmitted successfully.
```

RESULT:

Thus the sliding window protocol was implemented and the transmission of 8 frames from the sender to the receiver was simulated successfully. The lost frame 3 was retransmitted after the timeout and all frames were delivered in order.

Exp No: 8	NMAP to Discover Live Hosts using Nmap Scans (ARP, ICMP, TCP/UDP) on the TryHackMe Platform

AIM:

To use the Nmap tool on the TryHackMe platform to discover live hosts in a network using ARP, ICMP, TCP and UDP ping scans.

ALGORITHM / PROCEDURE:

1. Log in to the TryHackMe website, open the ***Nmap Live Host Discovery*** room and click ***Start AttackBox***. Click ***Start Machine*** to deploy the target.
2. Open the terminal in the AttackBox and note the IP address of the AttackBox using ip a.
3. ARP scan: run nmap -PR -sn 10.10.5.0/24. ARP works only for hosts on the same subnet.
4. ICMP scan: run nmap -PE -sn 10.10.5.0/24 to send ICMP echo requests (ping).
5. TCP scan: run nmap -PS22,80,443 -sn (SYN ping) and nmap -PA80 -sn (ACK ping).
6. UDP scan: run nmap -PU53,161 -sn to send empty UDP packets to the given ports.
7. Note the hosts reported as "up" and compare the results of the four scan types.

Fig 1: Host discovery probes sent from the AttackBox

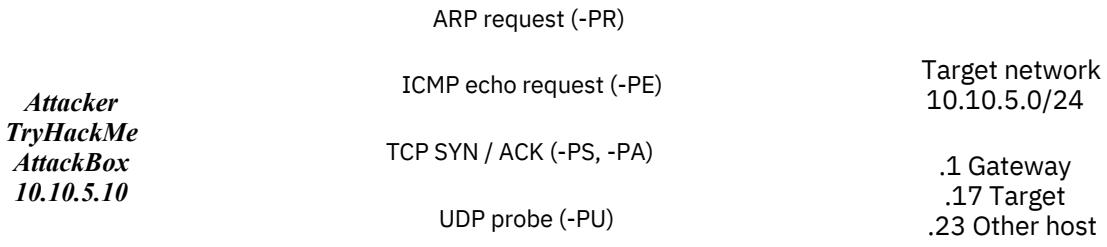
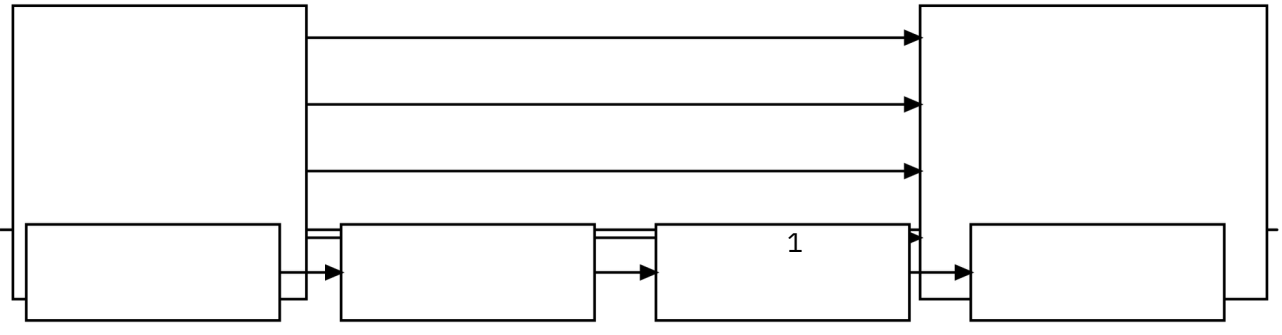


Fig 2: Working of a host discovery scan





Commands used:

Scan type	*Option*	*Command*
ARP ping	-PR	nmap -PR -sn 10.10.5.0/24
ICMP echo ping	-PE	nmap -PE -sn 10.10.5.0/24
TCP SYN ping	-PS	nmap -PS22,80,443 -sn 10.10.5.0/24
TCP ACK ping	-PA	nmap -PA80 -sn 10.10.5.0/24
UDP ping	-PU	nmap -PU53,161 -sn 10.10.5.0/24
List scan (no probes)	-sL	nmap -sL 10.10.5.0/24

OUTPUT:

AttackBox terminal - ARP scan

```
root@ip-10-10-5-10:~# nmap -PR -sn 10.10.5.0/24
Starting Nmap 7.80 ( https://nmap.org ) at 2026-10-07 10:42 BST
Nmap scan report for 10.10.5.1
Host is up (0.00031s latency).
MAC Address: 02:2B:6C:41:9A:01 (Unknown)
Nmap scan report for 10.10.5.17
Host is up (0.00045s latency).
MAC Address: 02:7E:D3:08:5C:17 (Unknown)
Nmap scan report for 10.10.5.23
Host is up (0.00052s latency).
MAC Address: 02:19:A4:F2:70:23 (Unknown)
Nmap scan report for 10.10.5.10
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.87 seconds
```

AttackBox terminal - ICMP echo scan

```
root@ip-10-10-5-10:~# nmap -PE -sn 10.10.5.0/24
Starting Nmap 7.80 ( https://nmap.org ) at 2026-10-07 10:44 BST
Nmap scan report for 10.10.5.1
Host is up (0.00029s latency).
MAC Address: 02:2B:6C:41:9A:01 (Unknown)
Nmap scan report for 10.10.5.17
Host is up (0.00041s latency).
MAC Address: 02:7E:D3:08:5C:17 (Unknown)
Nmap scan report for 10.10.5.10
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 3.11 seconds
```

AttackBox terminal - TCP SYN ping

```
root@ip-10-10-5-10:~# nmap -PS22,80,443 -sn 10.10.5.0/24
Starting Nmap 7.80 ( https://nmap.org ) at 2026-10-07 10:46 BST
Nmap scan report for 10.10.5.17
Host is up (0.00038s latency).
MAC Address: 02:7E:D3:08:5C:17 (Unknown)
Nmap scan report for 10.10.5.23
Host is up (0.00049s latency).
MAC Address: 02:19:A4:F2:70:23 (Unknown)
Nmap scan report for 10.10.5.10
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 3.04 seconds
```

AttackBox terminal - TCP ACK ping

```
root@ip-10-10-5-10:~# nmap -PA80 -sn 10.10.5.0/24
Starting Nmap 7.80 ( https://nmap.org ) at 2026-10-07 10:47 BST
Nmap scan report for 10.10.5.17
Host is up (0.00036s latency).
MAC Address: 02:7E:D3:08:5C:17 (Unknown)
Nmap scan report for 10.10.5.10
Host is up.
Nmap done: 256 IP addresses (2 hosts up) scanned in 3.02 seconds
```

AttackBox terminal - UDP ping

```
root@ip-10-10-5-10:~# nmap -PU53,161 -sn 10.10.5.0/24
Starting Nmap 7.80 ( https://nmap.org ) at 2026-10-07 10:49 BST
Nmap scan report for 10.10.5.17
Host is up (0.00044s latency).
MAC Address: 02:7E:D3:08:5C:17 (Unknown)
Nmap scan report for 10.10.5.10
Host is up.
Nmap done: 256 IP addresses (2 hosts up) scanned in 3.15 seconds
```

RESULT:

Thus the live hosts in the network were discovered using Nmap with ARP, ICMP, TCP and UDP scans on the TryHackMe platform. The ARP scan found the most hosts (4) because it works at the data link layer and cannot be blocked by a firewall on the same subnet, while the ICMP, TCP and UDP scans depended on how the target responded to each probe.

Exp No: 9	*Implementation of SUBNETTING in CISCO Packet Tracer Simulator*

AIM:

To design multiple subnets with a suitable number of hosts, assign static IP addresses, connect the subnets through a router in Cisco Packet Tracer, and observe packet transmission with and without the router.

ALGORITHM / PROCEDURE:

1. Take the network address 192.168.1.0/24 and decide the number of subnets needed (3 departments, so 4 subnets are created by borrowing 2 host bits).
2. The new mask is 255.255.255.192 (/26). Each subnet has 64 addresses, so 62 usable hosts.
3. Open Packet Tracer and place 1 router (2911), 3 switches (2960) and 6 PCs. Connect them with copper straight-through cables as in Fig 1.
4. Assign a static IP address, subnet mask and default gateway to every PC under ***Desktop > IP Configuration***.
5. `Configure the router interfaces: interface gig0/0, ip address 192.168.1.1 255.255.255.192, no shutdown`
6. Repeat step 5 for Gig0/1 and Gig0/2 with the gateway address of subnets B and C.
7. From the PC command prompt use ping to test connectivity inside a subnet and across subnets.
8. Remove the router (delete the links to the router) and ping across the subnets again to see the difference.

Fig 1: Network topology with three subnets joined by a router

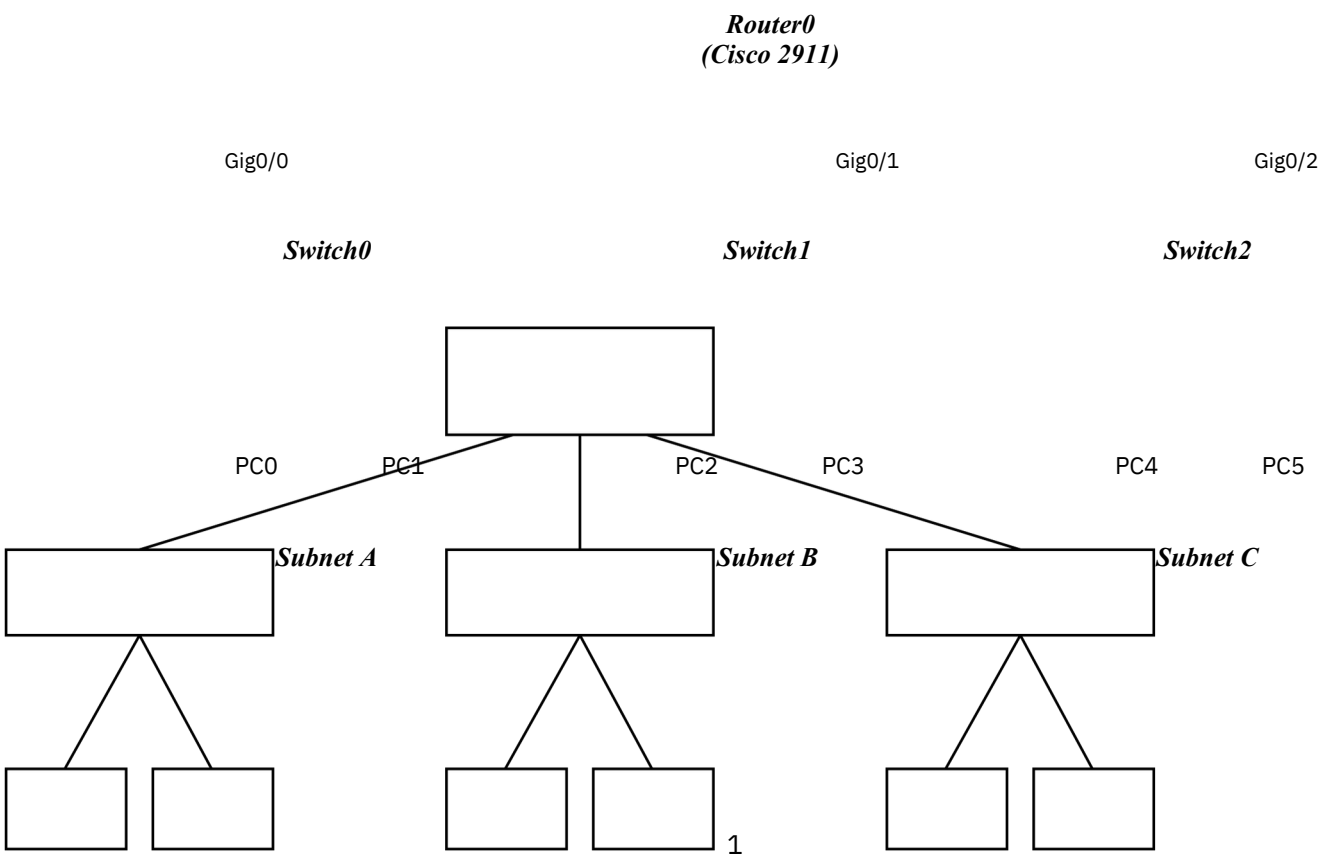


Fig 2: Borrowing 2 bits of the host part for subnetting

Subnet bits (borrowed)		*Host bits (6)*					
1	*1*	*h*	*h*	*h*	*h*	*h*	*h*

192.168.1.0 /24 -> /26(2 bits borrowed = 4 subnets, 2^6 - 2 = 62 hosts each)

Subnet design:

Subnet	*Network ID*	*Mask*	*Usable range*	*Broadcast*	*Gateway*
A (Sales)	192.168.1.0/26	255.255.255.1	.1 - .62	192.168.1.6	192.168.1.1
B (HR)	192.168.1.64/2	92	.65 - .126	3	192.168.1.65
C (IT)	6	255.255.255.1	.129 - .190	192.168.1.1	192.168.1.129
Unused	192.168.1.128/	92	.193 - .254	27	-
	26	255.255.255.1		192.168.1.1	
	192.168.1.192/	92		91	
	26	255.255.255.1		192.168.1.2	
		92		55	

IP address table:

Device	*Interface*	*IP Address*	*Subnet Mask*	*Default Gateway*
Router	Gig0/	192.168.1.1	255.255.255.1	-
0	0	192.168.1.6	92	-
Router	Gig0/	5	255.255.255.1	-
0	1	192.168.1.1	92	192.168.1.1
Router	Gig0/	29	255.255.255.1	192.168.1.1
0 PC0	2 Fa0	192.168.1.2	92	192.168.1.6
PC1	Fa0	192.168.1.3	255.255.255.1	5
PC2	Fa0	192.168.1.6	92	192.168.1.6
PC3	Fa0	6	255.255.255.1	5
PC4	Fa0	192.168.1.6	92	192.168.1.1
PC5	Fa0	7	255.255.255.1	29
		192.168.1.1	92	192.168.1.1
		30	255.255.255.1	29
		192.168.1.1	92	
		31	255.255.255.1	
			92	
			255.255.255.1	
			92	

Router configuration:

```
Router>enable
Router#configure terminal
Router(config)#interface gigabitEthernet 0/0
Router(config-if)#ip address 192.168.1.1 255.255.255.192
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface gigabitEthernet 0/1
Router(config-if)#ip address 192.168.1.65 255.255.255.192
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface gigabitEthernet 0/2
Router(config-if)#ip address 192.168.1.129 255.255.255.192
Router(config-if)#no shutdown
Router(config-if)#end
Router#write memory
```

OUTPUT:

Router0 CLI - show ip interface brief

```
Router#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.1.1     YES manual up                    up
GigabitEthernet0/1     192.168.1.65    YES manual up                    up
GigabitEthernet0/2     192.168.1.129   YES manual up                    up
```

PC0 Command Prompt - same subnet

```
PC>ping 192.168.1.3

Pinging 192.168.1.3 with 32 bytes of data:

Reply from 192.168.1.3: bytes=32 time=1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

(a) When the subnets are connected via a router

PC0 Command Prompt - ping PC2 (Subnet B) through the router

```
PC>ping 192.168.1.66

Pinging 192.168.1.66 with 32 bytes of data:

Request timed out.
Reply from 192.168.1.66: bytes=32 time=3ms TTL=127
Reply from 192.168.1.66: bytes=32 time=2ms TTL=127
Reply from 192.168.1.66: bytes=32 time=2ms TTL=127

Ping statistics for 192.168.1.66:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss)
```

PC0 Command Prompt - ping PC4 (Subnet C) through the router

```
PC>ping 192.168.1.130

Pinging 192.168.1.130 with 32 bytes of data:

Reply from 192.168.1.130: bytes=32 time=2ms TTL=127
```

PC0 Command Prompt - ping PC4 (Subnet C) through the router (contd.)

```
Reply from 192.168.1.130: bytes=32 time=1ms TTL=127
Reply from 192.168.1.130: bytes=32 time=1ms TTL=127
Reply from 192.168.1.130: bytes=32 time=2ms TTL=127

Ping statistics for 192.168.1.130:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

(b) When the subnets are not connected (router removed)

PC0 Command Prompt - ping PC2 (Subnet B) without the router

```
PC>ping 192.168.1.66

Pinging 192.168.1.66 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.1.66:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss)
```

PC0 Command Prompt - ping PC1 (same subnet) without the router

```
PC>ping 192.168.1.3

Pinging 192.168.1.3 with 32 bytes of data:

Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

OBSERVATION:

Case	*Source -> Destination*	*Result*
Router connected	PC0 -> PC1 (same subnet)	Success, TTL = 128
Router connected	PC0 -> PC2 / PC4 (other subnet)	Success, TTL = 127 (one router hop). First packet lost while ARP is resolved.
Router removed	PC0 -> PC1 (same subnet)	Success, switch forwards the frames
Router removed	PC0 -> PC2 / PC4 (other subnet)	Fails, 100% loss. No device can route between subnets.

RESULT:

Thus subnetting of the 192.168.1.0/24 network into /26 subnets was implemented in Cisco Packet Tracer. Hosts in different subnets could communicate only when a router was connected between them, while hosts inside the same subnet could communicate even without the router.

Exp No: 10	*Internetworking with routers in CISCO PACKET TRACER simulator*

AIM:

To design and configure internetworks in Cisco Packet Tracer using (a) a router with static IP addresses, (b) a wireless router with DHCP server and an internet cloud, and (c) a switch, a router and Ethernet cables in the lab, and to trace how the packets are routed.

ALGORITHM / PROCEDURE:

(a) Simple internetwork using a router

1. Place a Router (2911), two Switches and six PCs (3 in each network) on the workspace.
2. Connect each PC to its switch and each switch to the router with ***copper straight-through cables***. Router interface Gig0/0 goes to Switch0 and `Gig0/1` goes to Switch1.
3. Assign the static IP addresses to the PCs (network 1: `192.168.1.0/24`, network 2: `192.168.2.0/24`) and set the ***default gateway*** of each PC to the router interface address (`192.168.1.1` / `192.168.2.1`).
4. Open the router CLI and configure both interfaces with ip address and `no shutdown` (commands given below).
5. Ping from PC0 to PC3 and check the reply. Use tracert to see the path.
6. Go to Simulation mode, add a simple PDU from PC0 to PC3 and trace the packet through Switch0, Router0 and Switch1.

(b) Wireless router, DHCP server and internet cloud

1. Place a Wireless Router, a Cable Modem, a Cloud, a Server, one PC (wired), a Laptop and a Smartphone.
2. Connect PC0 to a LAN port of the wireless router, the Internet port of the router to the Cable Modem, the modem to the Cloud and the Cloud to the Server (Coaxial / Ethernet).
3. Open the wireless router ***GUI***: set ***SSID = LabNet***, ***WPA2-PSK password*** and ***Internet setup = DHCP***. Set the ***DHCP server = Enabled*** (start IP 192.168.0.100, 50 users).
4. Give the server a static IP 203.0.113.10. On the Laptop and the Smartphone configure the ***Wireless0*** as SSID LabNet with the same password.
5. In IP Configuration select ***DHCP*** on the PC, Laptop and Smartphone and check the address received.
6. Ping the server and also open its web page from the browser of the Laptop.

(c) Inter-network in the lab using switch, router and Ethernet cables

1. Take one router, two switches, six PCs and eight straight-through UTP cables (made in
2. Experiment 2). Connect PCs to the switches, and the switches to the two Gigabit ports of the router. Check that all the link LEDs glow.
3. Assign the addresses 172.16.1.0/24 and 172.16.2.0/24 with the router as the default gateway and configure the router interfaces.
4. Ping within the same network and between the two networks. Note down the results.

Connection	***Cable used***
PC to Switch	Straight-through
Switch to Router	Straight-through
PC to Router (direct)	Crossover

Connection	*Cable used*
Router to Router (LAN side)	Crossover

Router configuration (Part a)

Router CLI

```
Router>enable
Router#configure terminal
Router(config)#hostname R1
R1(config)#interface gigabitEthernet 0/0
R1(config-if)#ip address 192.168.1.1 255.255.255.0
R1(config-if)#no shutdown
R1(config-if)#exit
R1(config)#interface gigabitEthernet 0/1
R1(config-if)#ip address 192.168.2.1 255.255.255.0
R1(config-if)#no shutdown
R1(config-if)#end
R1#copy running-config startup-config
```

DIAGRAM:

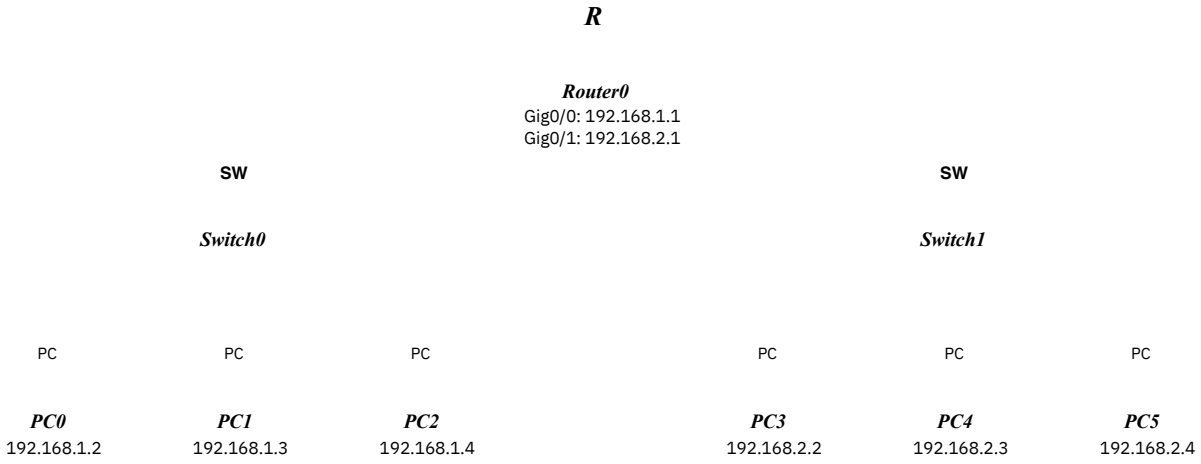


Fig 1: Internetwork of two LANs connected by a router (Part a)

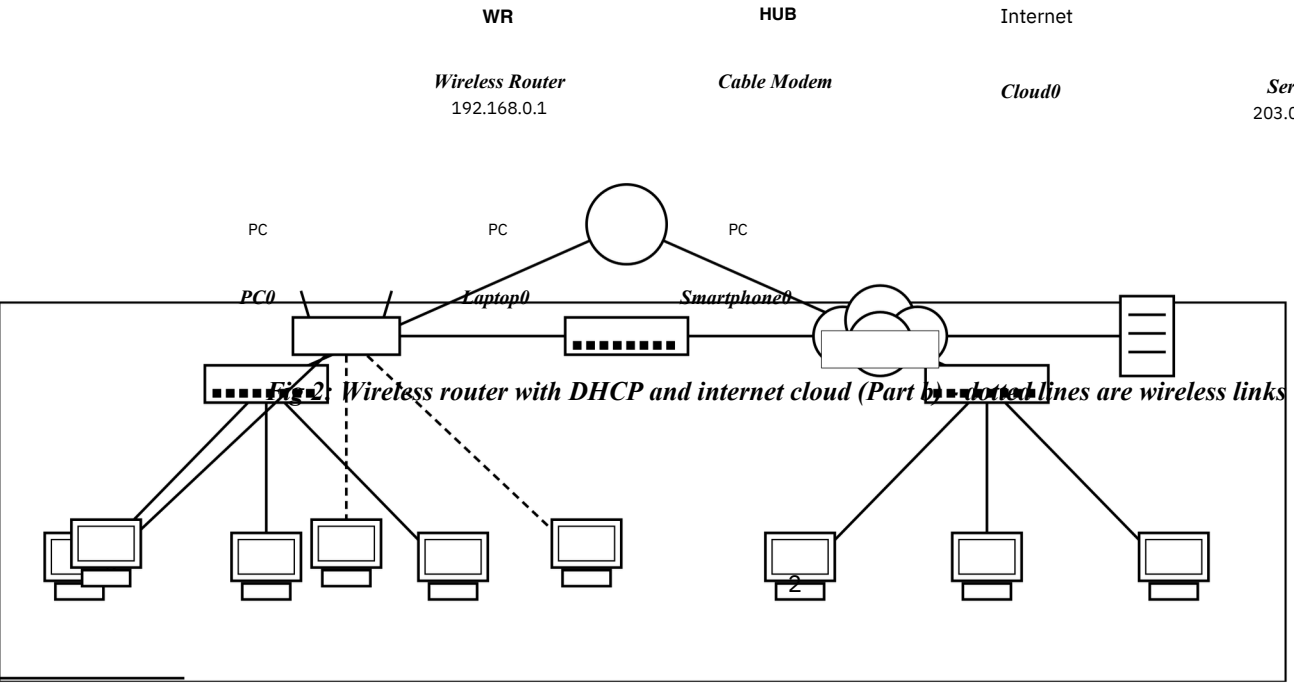


Fig 2: Wireless router with DHCP and internet cloud (Part b) – dotted lines are wireless links

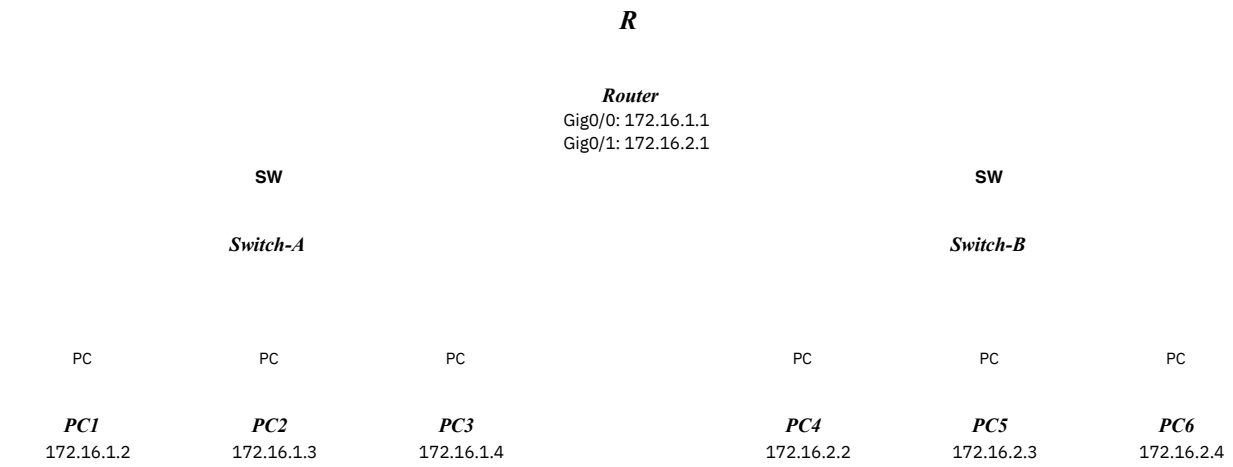
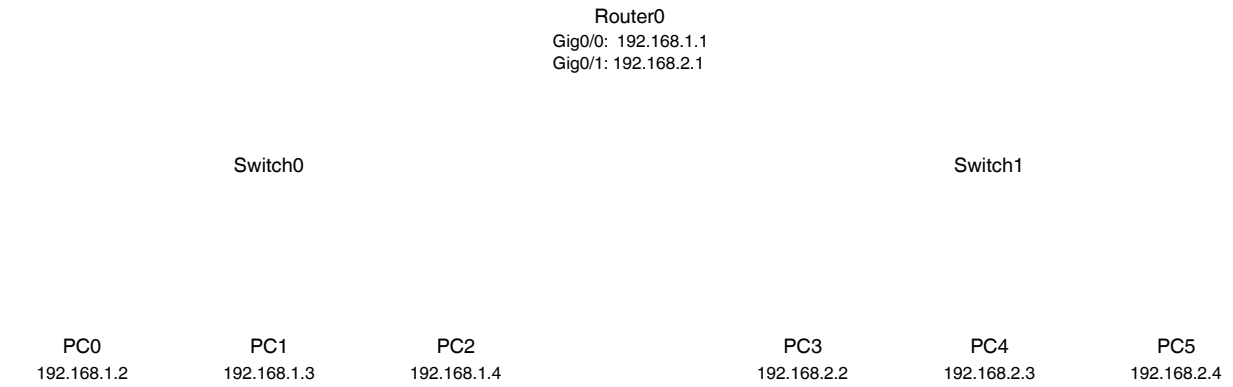


Fig 3: Lab inter-network using switches, router and Ethernet cables (Part c)

OUTPUT:

Part (a) - Router with two networks



Packet Tracer view of Part (a)

R1 - CLI

```
                no shutdown

%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up
R1(config-if)#end
R1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.1.1     YES manual up                    up
GigabitEthernet0/1     192.168.2.1     YES manual up                    up
Vlan1                  unassigned      YES unset administratively down down
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP

Gateway of last resort is not set

     192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.1.0/24 is directly connected, GigabitEthernet0/0
L       192.168.1.1/32 is directly connected, GigabitEthernet0/0
     192.168.2.0/24 is variably subnetted, 2 subnets, 2 masks
```

R1 - CLI (contd.)

```
C       192.168.2.0/24 is directly connected, GigabitEthernet0/1
L       192.168.2.1/32 is directly connected, GigabitEthernet0/1
```

PC0 - Command Prompt

```
C:\>

Pinging 192.168.2.3 with 32 bytes of data:

Request timed out.
Reply from 192.168.2.3: bytes=32 time=2ms TTL=127
Reply from 192.168.2.3: bytes=32 time=1ms TTL=127
Reply from 192.168.2.3: bytes=32 time=1ms TTL=127

Ping statistics for 192.168.2.3:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 2ms, Average = 1ms

C:\>

Tracing route to 192.168.2.3 over a maximum of 30 hops:

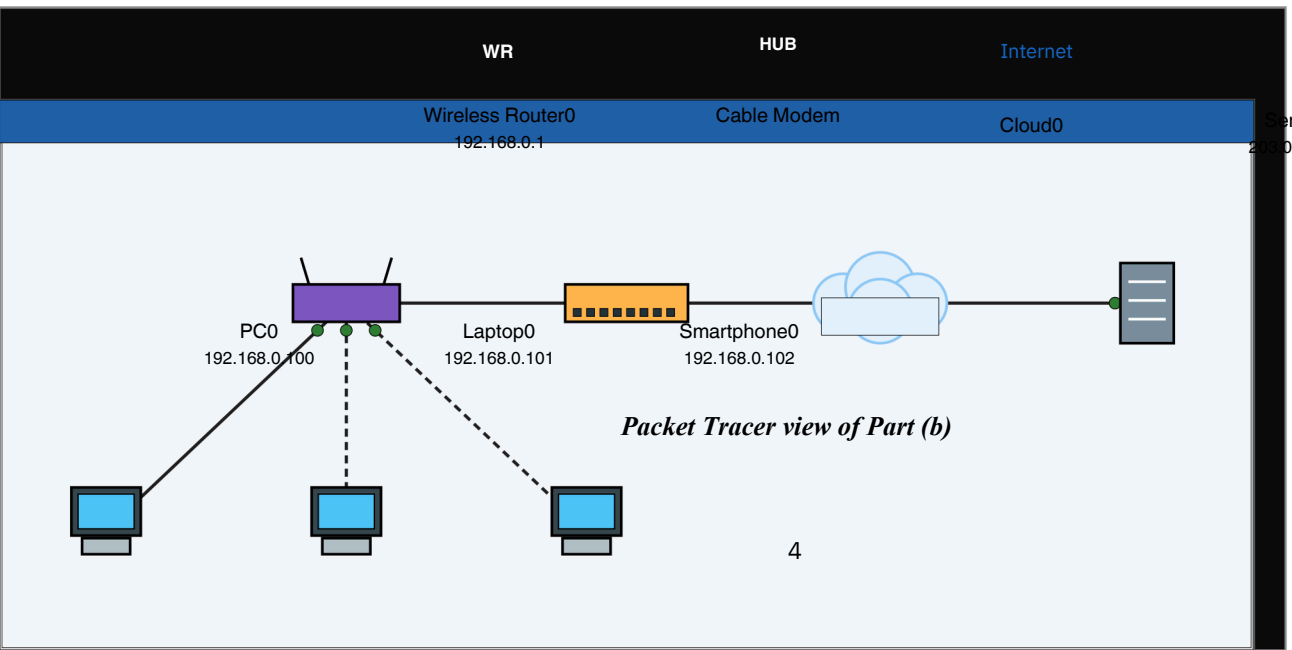
  1    0 ms      0ms       0 ms      192.168.1.1
  2    0 ms      0ms       1 ms      192.168.2.3

Trace complete.
```

Fire	Last Status	Source	Destination	Type	Color	Time(sec	Periodic	Num
	Successful	PC0	Switch0	ICMP		) 0.000	N	0
	Successful	Switch0	Router0	ICMP		0.001	N	1
	Successful	Router0	Switch1	ICMP		0.002	N	2
	Successful	Switch1	PC3	ICMP		0.003	N	3
	Successful	PC3	Switch1	ICMP		0.004	N	4
	Successful	Switch1	Router0	ICMP		0.005	N	5
	Successful	Router0	Switch0	ICMP		0.006	N	6
	Successful	Switch0	PC0	ICMP		0.007	N	7

The router receives the frame on Gig0/0, removes the Ethernet header, looks at the destination IP in the routing table, and sends a new frame out of Gig0/1 to PC3.

Part (b) - Wireless router, DHCP and internet cloud



Packet Tracer view of Part (b)

Laptop0 - Command Prompt

```
C:\>

Wireless0 Connection:(default port)

   Connection-specific DNS Suffix..:
   Physical Address.................: 0001.C7AB.4D12 Link-local
   IPv6   Address.........:   FE80::201:C7FF:FEAB:4D12   IPv6
   Address.....................: ::
   IP Address.....................: 192.168.0.101
   Subnet Mask.....................: 255.255.255.0
   Default Gateway.................: 192.168.0.1
   DHCP Servers....................: 192.168.0.1
   DNS Servers.....................: 0.0.0.0

C:\>

Pinging 203.0.113.10 with 32 bytes of data:

Reply from 203.0.113.10: bytes=32 time=21ms TTL=126
Reply from 203.0.113.10: bytes=32 time=12ms TTL=126
Reply from 203.0.113.10: bytes=32 time=14ms TTL=126
Reply from 203.0.113.10: bytes=32 time=11ms TTL=126

Ping statistics for 203.0.113.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 11ms, Maximum = 21ms, Average = 14ms
```

Device	*Connection*	*Method*	*IP address received*	*Ping to server*
PC0	Wired (LAN port)	DHCP	192.168.0.100	Successful
Laptop0	Wireless (LabNet)	DHCP	192.168.0.101	Successful
Smartphone0	Wireless (LabNet)	DHCP	192.168.0.102	Successful



Part (c) - Lab network with switches, router and Ethernet cables

PC1 - Command Prompt

```
C:\>

FastEthernet0 Connection:(default port)

   IP Address.....................: 172.16.1.2
   Subnet Mask....................: 255.255.255.0
   Default Gateway................: 172.16.1.1

C:\>

Pinging 172.16.1.3 with 32 bytes of data:

Reply from 172.16.1.3: bytes=32 time=1ms TTL=127
Reply from 172.16.1.3: bytes=32 time=<1ms TTL=127
Reply from 172.16.1.3: bytes=32 time=<1ms TTL=127
Reply from 172.16.1.3: bytes=32 time=<1ms TTL=127

Ping statistics for 172.16.1.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 2ms, Average = 1ms

C:\>

Pinging 172.16.2.4 with 32 bytes of data:

Request timed out.
Reply from 172.16.2.4: bytes=32 time=2ms TTL=127
Reply from 172.16.2.4: bytes=32 time=1ms TTL=127
Reply from 172.16.2.4: bytes=32 time=1ms TTL=127

Ping statistics for 172.16.2.4:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 2ms, Average = 1ms
```

Source	*Destination*	*Same network?*	*Through router?*	*Result*
PC1 (172.16.1.2)	PC3 (172.16.1.4)	Yes No	No Yes	Successful
PC1 (172.16.1.2)	PC4 (172.16.2.2)	No	Yes	Successful
PC1	PC2			Successf

RESULT:

Thus the internetworks were designed and configured in Cisco Packet Tracer using a router with static IP addresses, a wireless router with DHCP server and internet cloud, and a lab network using switches, a router and Ethernet cables. The packet transmission across the networks was traced in simulation mode.

Exp No: 11	*Routing at Network Layer - Static Routing and RIP*

AIM:
To simulate static routing and the RIP dynamic routing protocol configuration using Cisco Packet Tracer.

ALGORITHM / PROCEDURE:

(a) Static routing

1. Place 3 routers (2911 with HWIC-2T serial card) and 2 PCs. Connect PC0 to R1 and PC1 to R3 with a crossover cable, and connect the routers using ***serial DCE*** cables (R1-R2 and R2-R3).
2. Assign IP addresses to all the router interfaces and the PCs as shown in the address table. Set the default gateways (192.168.1.1 and 192.168.2.1).
3. On the DCE side of each serial link give clock rate 64000 and use no shutdown on all interfaces.
4. Ping from PC0 to PC1. It fails because the routers do not know the remote networks.
5. On each router add a static route for every network that is not directly connected: ip route <network> <mask> <next-hop>
6. Verify with show ip route and ping PC0 to PC1 again. Use tracert to see the path.

(b) RIP (Routing Information Protocol)

1. Remove the static routes using no ip route ... (or build the same topology again).
2. On every router enter RIP configuration mode with router rip and version 2.
3. Advertise the directly connected networks using the network <address> command and disable auto summary using no auto-summary.
4. Wait for about 30 seconds (RIP update timer). Check the learnt routes with show ip route and show ip protocols.
5. Ping PC0 to PC1 and trace the path.

Device	***Interface***	***IP address***	***Subnet mask***
R1	Gig0/0	192.168.1.1	255.255.255.0
R1	Se0/0/0 (DCE)	10.0.0.1	255.255.255.252
R2	Se0/0/0	10.0.0.2	255.255.255.252
R2	Se0/0/1 (DCE)	10.0.0.5	255.255.255.252
R3	Se0/0/1	10.0.0.6	255.255.255.252
R3	Gig0/0	192.168.2.1	255.255.255.0
PC0	NIC	192.168.1.2	255.255.255.0
PC1	NIC	192.168.2.2	255.255.255.0

Commands used

Static routing	*RIP*
R1(config)#ip route 192.168.2.0 255.255.255.0 10.0.0.2	R1(config)#router rip
R1(config)#ip route 10.0.0.4 255.255.255.252 10.0.0.2	R1(config-router)#version 2
R2(config)#ip route 192.168.1.0 255.255.255.0 10.0.0.1	R1(config-router)#network 192.168.1.0
R2(config)#ip route 192.168.2.0 255.255.255.0 10.0.0.6	R1(config-router)#network 10.0.0.0
R3(config)#ip route 192.168.1.0 255.255.255.0 10.0.0.5	R1(config-router)#no auto-summary
R3(config)#ip route 10.0.0.0 255.255.255.252 10.0.0.5	(similar for R2: 10.0.0.0 and R3: 192.168.2.0)

DIAGRAM:

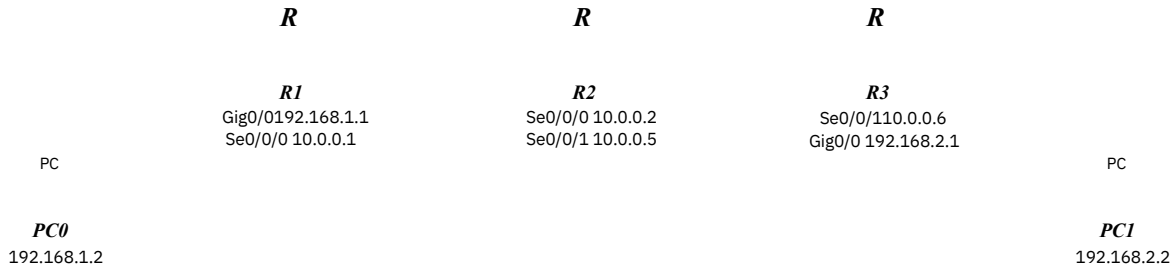
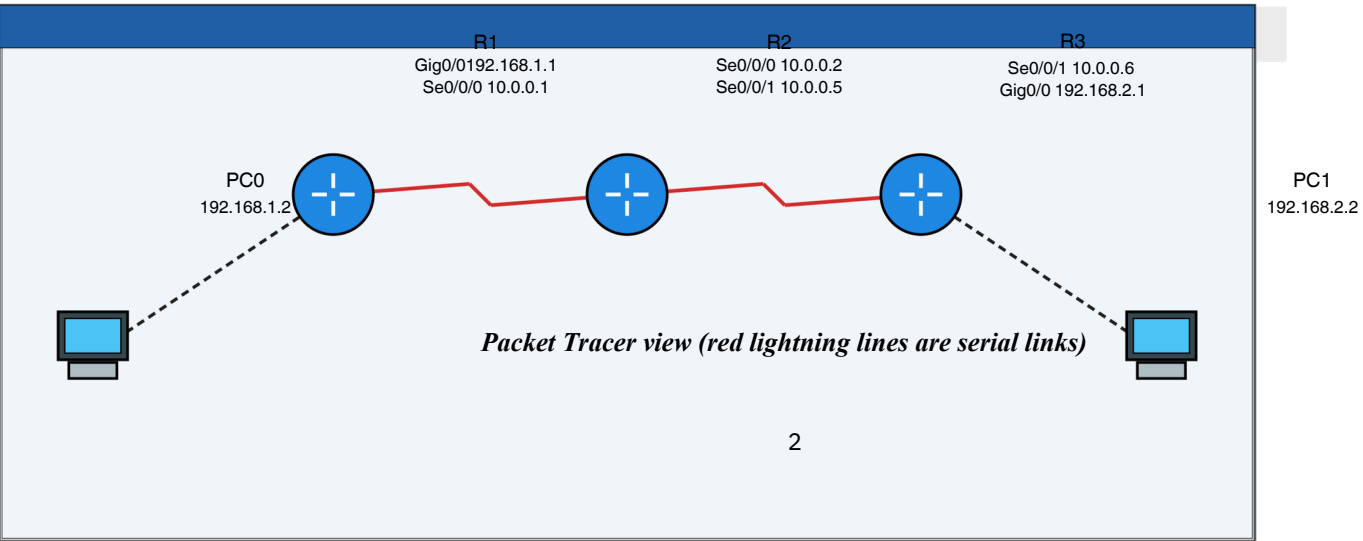


Fig 1: Three routers connected by serial links (dotted lines are serial links)

OUTPUT:

Part (a) - Static routing



PC0 - Command Prompt (before static routes)

```
C:\>

Pinging 192.168.2.2 with 32 bytes of data:

Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.

Ping statistics for 192.168.2.2:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

R1 - CLI

```
   show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP

Gateway of last resort is not set

     10.0.0.0/30 is subnetted, 2 subnets
C       10.0.0.0 is directly connected, Serial0/0/0
S       10.0.0.4 [1/0] via 10.0.0.2
C    192.168.1.0/24 is directly connected, GigabitEthernet0/0
S    192.168.2.0/24 [1/0] via 10.0.0.2
```

PC0 - Command Prompt (after static routes)

```
C:\>

Pinging 192.168.2.2 with 32 bytes of data:

Request timed out.
Reply from 192.168.2.2: bytes=32 time=3ms TTL=125
Reply from 192.168.2.2: bytes=32 time=2ms TTL=125
Reply from 192.168.2.2: bytes=32 time=2ms TTL=125

Ping statistics for 192.168.2.2:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),

C:\>

Tracing route to 192.168.2.2 over a maximum of 30 hops:

  1    0ms       0ms       0ms       192.168.1.1
  2    1ms       1ms       0ms       10.0.0.2
  3    2ms       1ms       2ms       10.0.0.6
  4    2ms       2ms       1ms       192.168.2.2

Trace complete.
```

Part (b) - RIP

R1 - CLI

```
router rip
 version 2
 network 192.168.1.0
 network 10.0.0.0
 no auto-summary
 end
show ip route rip
  10.0.0.0/30 is subnetted, 2 subnets
R       10.0.0.4 [120/1] via 10.0.0.2, 00:00:12, Serial0/0/0
R    192.168.2.0/24 [120/2] via 10.0.0.2, 00:00:12, Serial0/0/0
show ip protocols
Routing Protocol is "rip"
  Sending updates every 30 seconds, next due in 17 seconds
Invalid after 180 seconds, hold down 180, flushed after 240
Outgoing update filter list for all interfaces is not set
Incoming update filter list for all interfaces is not set
Redistributing: rip
  Default version control: send version 2, receive 2
Automatic network summarization is not in effect
 Maximum path: 4
 Routing for Networks:
   10.0.0.0
   192.168.1.0
 Routing Information Sources:
   Gateway         Distance      Last Update
   10.0.0.2             120      00:00:12
 Distance: (default is 120)
```

PC0 - Command Prompt

```
C:\>

Pinging 192.168.2.2 with 32 bytes of data:

Reply from 192.168.2.2: bytes=32 time=2ms TTL=125
Reply from 192.168.2.2: bytes=32 time=2ms TTL=125
Reply from 192.168.2.2: bytes=32 time=1ms TTL=125
Reply from 192.168.2.2: bytes=32 time=2ms TTL=125

Ping statistics for 192.168.2.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
```

Fire	Last Status	Source	Destination	Type	Color	Time(sec	Periodic	Num
	Successful	PC0 R1	R1	ICMP		) 0.000	N	0
	Successful	R2	R2	ICMP		0.001	N	1
	Successful							
	Successful							
	Successful							
	Successful							

		neighbours
Updates	None	Every 30 seconds
Link failure	Admin has to change the route	Finds another path automatically
Metric	Not used	Hop count (max 15)
Best for	Small and stable networks	Small to medium networks

RESULT:

Thus static routing and RIP were configured and simulated in Cisco Packet Tracer. Ping between the two remote networks failed without routes and became successful after configuring the static routes and also after configuring RIP.

Exp No: 12	*End-End Communication at Transport Layer - Echo Client Server and Chat*

AIM:

To implement (a) an echo client-server using TCP and UDP sockets and (b) a chat program using socket programming, for end-to-end communication at the transport layer.

ALGORITHM / PROCEDURE:

(a1) TCP echo server and client

1. ***Server***: create a TCP socket with socket(AF_INET, SOCK_STREAM), bind() it to an IP address and port (5000), call listen() and wait in accept() for the client.
2. ***Client***: create a TCP socket and call connect() with the IP address and port of the server.
3. The client sends a message using send(). The server reads it with recv(), displays it and sends the same data back using sendall().
4. The client receives the echoed message and displays it. This is repeated for each message.
5. Close the sockets using close().

(a2) UDP echo server and client

1. ***Server***: create a UDP socket with SOCK_DGRAM and bind() it to port 6000. No `listen()` / `accept()` is needed because UDP is connectionless.
2. ***Client***: create a UDP socket and send the message to the server address with sendto().
3. The server receives the datagram and the client address with recvfrom() and sends it back using `sendto()`.
4. The client receives the echo with recvfrom() and displays it. Close the sockets.

(b) Chat program

1. The chat server creates a TCP socket on port 7000 and waits for a client. The client connects to it.
2. The client types a message and sends it. The server displays it, then the server user types the reply and sends it back.
3. Messages are exchanged alternately until either side sends the word exit. Then both sockets are closed.

Point	***TCP***	***UDP***
Socket type	SOCK_STREAM	SOCK_DGRAM
Connection	Connection oriented (3-way handshake)	Connectionless
Reliability	Reliable, ordered delivery	No guarantee
Server calls	bind, listen, accept, recv, send	bind, recvfrom, sendto
Used in	HTTP, FTP, SMTP	DNS, DHCP, streaming

DIAGRAM:

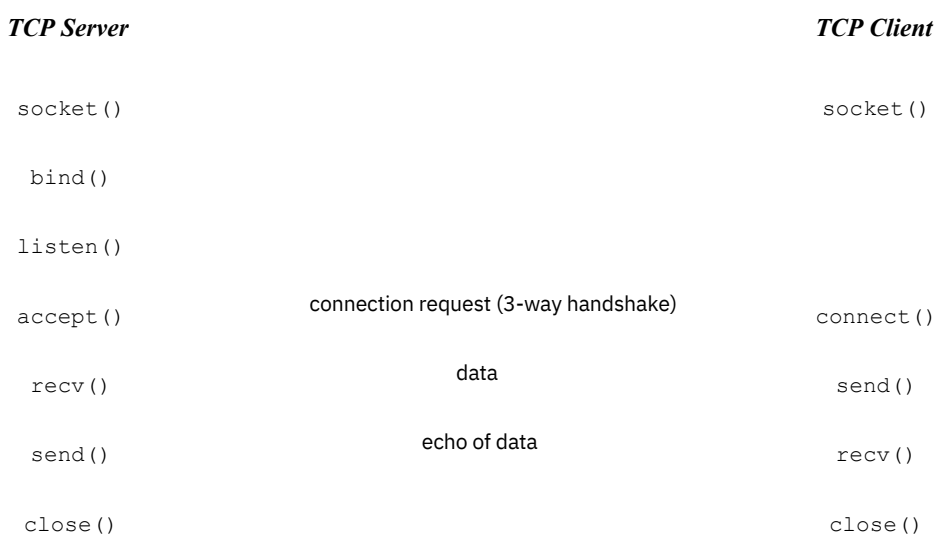


Fig 1: Socket calls in TCP client-server communication



Fig 2: Socket calls in UDP client-server communication

PROGRAM:

echo_tcp_server.py

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(("127.0.0.1", 5000))
s.listen(1)
print("TCP Echo Server listening on port 5000 ...")
conn, addr = s.accept()
print("Connected by", addr)
while True:
    data = conn.recv(1024)
    if not data:
        break
    print("Received :", data.decode())
    conn.sendall(data)                # echo back
conn.close()
s.close()
print("Connection closed.")
```

echo_tcp_client.py

```
import socket
c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(("127.0.0.1", 5000))
for msg in ["Hello Server", "Computer Networks Lab", "bye"]:
```

echo_tcp_client.py (contd.)

```
    c.sendall(msg.encode())
    print("Sent     :", msg)
    print("Echoed   :", c.recv(1024).decode())
c.close()
```

echo_udp_server.py

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(("127.0.0.1", 6000))
print("UDP Echo Server waiting on port 6000 ...")
for _ in range(2):
    data, addr = s.recvfrom(1024)
    print("Received from", addr, ":", data.decode())
    s.sendto(data, addr)
s.close()
```

echo_udp_client.py

```
import socket
c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
for msg in ["Hello UDP", "Datagram test"]:
    c.sendto(msg.encode(), ("127.0.0.1", 6000))
    data, _ = c.recvfrom(1024)
    print("Sent     :", msg)
    print("Echoed   :", data.decode())
c.close()
```

chat_server.py

```
import socket
s = socket.socket()
s.bind(("127.0.0.1", 7000))
s.listen(1)
print("Chat server started. Waiting for client ...")
conn, addr = s.accept()
print("Client connected:", addr)
while True:
    msg = conn.recv(1024).decode()
    if not msg or msg.lower() == "exit":
        break
    print("Client :", msg)
    reply = input("Server : ")
    conn.send(reply.encode())
    if reply.lower() == "exit":
        break
conn.close()
s.close()
print("Chat ended.")
```

chat_client.py

```
import socket
c = socket.socket()
c.connect(("127.0.0.1", 7000))
print("Connected to chat server. Type 'exit' to quit.")
while True:
    msg = input("Client : ")
    c.send(msg.encode())
    if msg.lower() == "exit":
        break
    reply = c.recv(1024).decode()
    if reply.lower() == "exit":
        break
    print("Server :", reply)
c.close()
print("Chat ended.")
```

OUTPUT:

(a1) TCP echo

UusSeErR@UusSeErR-VirtualBox: ~ (Terminal 1 - Server)

```
UusSeErR@UusSeErR-VirtualBox:~$
TCP Echo Server listening on port 5000 ...
Connected by ('127.0.0.1', 46248)
Received : Hello Server
Received : Computer Networks Lab
Received : bye
Connection closed.
```

UusSeErR@UusSeErR-VirtualBox: ~ (Terminal 2 - Client)

```
UusSeErR@UusSeErR-VirtualBox:~$
Sent     : Hello Server
Echoed   : Hello Server
Sent     : Computer Networks Lab
Echoed   : Computer Networks Lab
Sent     : bye
Echoed   : bye
```

(a2) UDP echo

UusSeErR@UusSeErR-VirtualBox: ~ (Terminal 1 - Server)

```
UusSeErR@UusSeErR-VirtualBox:~$
UDP Echo Server waiting on port 6000 ...
Received from ('127.0.0.1', 53205) : Hello UDP
Received from ('127.0.0.1', 53205) : Datagram test
```

UusSeErR@UusSeErR-VirtualBox: ~ (Terminal 2 - Client)

```
UusSeErR@UusSeErR-VirtualBox:~$
Sent     : Hello     UDP
Echoed   : Hello     UDP
Sent     : Datagram test
Echoed   : Datagram test
```

(b) Chat program

UusSeErR@UusSeErR-VirtualBox: ~ (Terminal 1 - Chat Server)

```
UusSeErR@UusSeErR-VirtualBox:~$
Chat server started. Waiting for client ...
Client connected: ('127.0.0.1', 43260)
Client : Hi Server, this is UusSeErR
Server : Hello UusSeErR , how can I help?
Client : Is the CN lab record ready?
Server : Yes, the lab record is due on Friday.
Client : Thanks, bye
Server : exit
Chat ended.
```

UusSeErR@UusSeErR-VirtualBox: ~ (Terminal 2 - Chat Client)

```
UusSeErR@UusSeErR-VirtualBox:~$
Connected to chat server. Type 'exit' to quit.
Client : Hi Server, this is UusSeErR
Server : Hello UusSeErR , how can I help?
Client : Is the CN lab record ready?
Server : Yes, the lab record is due on Friday.
Client : Thanks, bye
Chat ended.
```

RESULT:

Thus the echo client-server programs using TCP and UDP sockets and the chat program using socket programming were implemented and executed successfully.

Exp No: 13	*Implement your own ping program*

AIM:

To implement our own ping program using raw sockets in Python, which sends ICMP Echo Request messages to a host, receives the ICMP Echo Replies and displays the round trip time and the packet loss.

ALGORITHM / PROCEDURE:

1. Read the host name given in the command line and convert it into an IP address using `socket.gethostbyname()`.
2. `Create a` ***`raw socket`*** `using socket(AF_INET, SOCK_RAW, IPPROTO_ICMP). Raw sockets need` administrator rights, so the program is run with sudo.
3. Build the ICMP Echo Request: ***Type*** = 8, ***Code*** = 0, ***Checksum*** = 0 (for now), ***Identifier*** = process id, ***Sequence number*** = 1, 2, 3 ... followed by the data (current time + 16 bytes of text).
4. Calculate the ***Internet checksum*** (16 bit one's complement sum of the whole ICMP message) and put it in the checksum field.
5. Send the packet with sendto() and note the time. Wait for the reply with recvfrom() (time out = 2 seconds).
6. The received data has a 20 byte IP header in front. Skip it, read the ICMP header and check that the ***type*** is 0 (Echo Reply) and the ***identifier*** is our id. Read the TTL from byte 8 of the IP header.
7. Round trip time = time of receiving - time of sending. Display the reply line. If no reply comes, display `Request timed out.`
8. Repeat steps 3 to 7 four times with a gap of one second.
9. Finally display the number of packets sent, received, % loss and the minimum / average / maximum round trip time.

Header and functions used

Item	*Meaning*
`struct.pack("!BBHHH", ...)`	Packs type, code, checksum, identifier and sequence into 8 bytes in network byte order
`socket.SOCK_RAW`	Gives direct access to the IP packets (we build the ICMP part ourselves)
`IPPROTO_ICMP`	Tells the kernel that this raw socket handles ICMP messages (IP protocol number 1)
`checksum()`	Adds all 16 bit words, adds the carry back and takes the one's complement
`data[8]`	TTL value inside the received IP header

DIAGRAM:

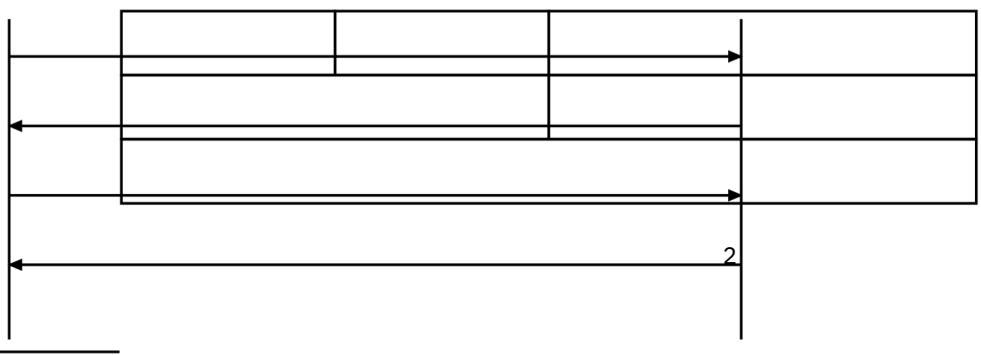
	0	8	16	31
Byte 0	Type (8)	Code (8)	Checksum (16)	
Byte 4	Identifier (16)		Sequence number (16)	
Byte 8	Data : timestamp (8 bytes) + payload "abcdefghijklmnop"			

Echo Request: Type = 8, Code = 0 ***Echo Reply: Type = 0, Code = 0***

Fig 1: ICMP Echo Request / Reply message format

My ping program	***Destination host***
ICMP Echo Request (type 8, seq 1)	
ICMP Echo Reply (type 0, seq 1)	
ICMP Echo Request (type 8, seq 2)	
ICMP Echo Reply (type 0, seq 2)	

Fig 2: Working of the ping program



PROGRAM:

myping.py

```
# Own ping program using RAW sockets (ICMP) - run as root
import socket, struct, time, sys, os

def checksum(data):
    if len(data) % 2:
        data += b'\x00'
    s = sum(struct.unpack("!%dH" % (len(data) // 2), data))
    s = (s >> 16) + (s & 0xffff)
    s += s >> 16
    return ~s & 0xffff

def make_packet(seq, pid):
    header = struct.pack("!BBHHH", 8, 0, 0, pid, seq)      # type 8 = echo request
    payload = struct.pack("!d", time.time()) + b"abcdefghijklmnop"
    chk = checksum(header + payload)
    header = struct.pack("!BBHHH", 8, 0, chk, pid, seq)
    return header + payload

def ping(host, count=4):
    try:
        ip = socket.gethostbyname(host)
    except socket.gaierror:
        print(f"ping: {host}: Name or service not known")
        return
    s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP)
    s.settimeout(2)
    pid = os.getpid() & 0xffff
    print(f"PING {host} ({ip}) 32 bytes of data.")
    times = []
    for seq in range(1, count + 1):
        s.sendto(make_packet(seq, pid), (ip, 0))
        t1 = time.time()
        try:
            while True:
                data, addr = s.recvfrom(1024)
                typ, code, _, rid, rseq = struct.unpack("!BBHHH", data[20:28])
                if typ == 0 and rid == pid:
                    break
            rtt = (time.time() - t1) * 1000
            ttl = data[8]
            times.append(rtt)
            print(f"{len(data)-20} bytes from {addr[0]}: icmp_seq={rseq} ttl={ttl} time={rtt:.3f} ms
              ")
        except socket.timeout:
            print(f"Request timed out for icmp_seq={seq}")
        time.sleep(1)
    s.close()
    lost = count - len(times)
    print(f"\n--- {host} ping statistics ---")
    print(f"{count} packets transmitted, {len(times)} received, {lost*100//count}% packet loss")
    if times:
        print(f"rtt min/avg/max = {min(times):.3f}/{sum(times)/len(times):.3f}/{max(times):.3f} ms")

ping(sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1")
```

OUTPUT:

```
UusSeErR@UusSeErR-VirtualBox: ~
UusSeErR@UusSeErR-VirtualBox:~$
PING 127.0.0.1 (127.0.0.1) 32 bytes of data.
32 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.015 ms
32 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.017 ms
32 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.023 ms
32 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.036 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
rtt min/avg/max = 0.015/0.023/0.036 ms
UusSeErR@UusSeErR-VirtualBox:~$
ping: abc.invalid: Name or service not known
```

Quantity	*Value*
Packets transmitted	4
Packets received	4
Packet loss	0%
Minimum / average / maximum RTT	0.015 / 0.023 / 0.036 ms
TTL of reply	64 (loopback address)

RESULT:

Thus our own ping program was implemented using raw sockets. ICMP Echo Request packets were sent, the Echo Replies were received and the round trip time and packet loss statistics were displayed successfully.

Exp No: 14	*Packet sniffing using RAW sockets*

AIM:

To write a program using raw sockets to capture (sniff) the packets passing through a network interface and to display the Ethernet, IP and TCP / UDP / ICMP header details.

ALGORITHM / PROCEDURE:

1. Create a raw socket with socket(AF_PACKET, SOCK_RAW, ntohs(3)). This socket receives every frame (ETH_P_ALL = 3) that reaches the network card, with the Ethernet header. Root permission is required.
2. Start an infinite loop and receive a frame with recvfrom() (maximum 65535 bytes).
3. Unpack the first 14 bytes as the ***Ethernet header***: destination MAC (6 bytes), source MAC (6 bytes) and the type field (2 bytes).
4. If the type is 0x0800 (IPv4), unpack the next 20 bytes as the ***IP header*** and get the TTL, the protocol number, the source IP and the destination IP. The header length is found from the IHL field.
5. Using the protocol number, unpack the transport header: 1 = ICMP (type, code), 6 = TCP (ports, sequence, acknowledgement), 17 = UDP (ports, length).
6. Display the details of the packet. Repeat until the required number of packets (8) is captured.
7. Generate some traffic in another terminal (ping, UDP echo, browser) and watch the packets being displayed. Close the socket.

Protocol number	*Protocol*	*Header read by the program*
1	ICMP	Type, Code
6	TCP	Source port, destination port, sequence no., acknowledgement no.
17	UDP	Source port, destination port, length

DIAGRAM:

Ethernet header	*IP header*	*TCP / UDP / ICMP header*	*Data (payload)*
14 bytes	20 bytes	20 / 8 / 8 bytes	remaining bytes
`struct "!6s6sH"`	`struct "!BBHHHBBH4s4s"`	`TCP "!HHLLB" / UDP "!HHH"`	

Packet received by the raw socket (AF_PACKET)

Fig 1: Structure of a captured frame and the struct format used for each header

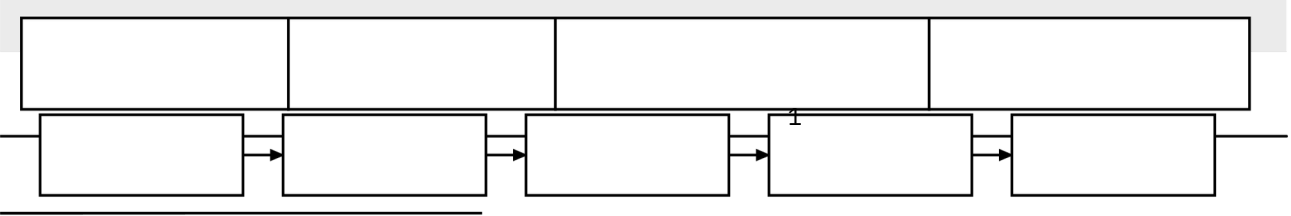


Fig 2: Flow of packet sniffing using a raw socket

PROGRAM:

sniffer.py

```
# Packet sniffer using RAW sockets (Linux) - run as root
import socket, struct

def mac(b):
    return ':'.join('%02x' % x for x in b)

s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.ntohs(3))
print("Sniffing started... (capturing 8 packets)\n")
count = 0
while count < 8:
    raw, info = s.recvfrom(65535)
    if info[2] == socket.PACKET_OUTGOING:      # skip our own outgoing copy
        continue
    dst, src, proto = struct.unpack("!6s6sH", raw[:14])
    if proto != 0x0800:                    # only IPv4
        continue
    count += 1
    ver_ihl, tos, tlen, ident, frag, ttl, p, chk, sip, dip = struct.unpack("!BBHHHBBH4s4s", raw[14:3
      4])
    ihl = (ver_ihl & 15) * 4
    print(f"Packet {count} [{info[0]}]")
    print(f"  Ethernet : {mac(src)} -> {mac(dst)}")
    print(f"  IPv4     : {socket.inet_ntoa(sip)} -> {socket.inet_ntoa(dip)} TTL={ttl} Length={tlen
      }")
    off = 14 + ihl
    if p == 1:
        t, c = struct.unpack("!BB", raw[off:off+2])
        print(f"  ICMP     : type={t} code={c}")
    elif p == 6:
        sp, dp, seq, ack, offs = struct.unpack("!HHLLB", raw[off:off+13])
        print(f"  TCP      : {sp} -> {dp} seq={seq} ack={ack}")
    elif p == 17:
        sp, dp, ln = struct.unpack("!HHH", raw[off:off+6])
        print(f"  UDP      : {sp} -> {dp} length={ln}")
    else:
        print(f"  Protocol : {p}")
    print()
s.close()
```

OUTPUT:

VirtualBox: ~ (Terminal 1)

```
UusSeErR@UusSeErR-VirtualBox:~$
Sniffing started... (capturing 8 packets)

Packet 1 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=37
  UDP      : 43740 -> 6000  length=17

Packet 2 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=37
  UDP      : 6000 -> 43740  length=17

Packet 3 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=41
  UDP      : 43740 -> 6000  length=21

Packet 4 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=41
  UDP      : 6000 -> 43740  length=21

Packet 5 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=52
  ICMP     : type=8 code=0

Packet 6 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=52
  ICMP     : type=0 code=0

Packet 7 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=52
  ICMP     : type=8 code=0

Packet 8 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=52
  ICMP     : type=0 code=0
```

VirtualBox: ~ (Terminal 2 - traffic generated)

```
UusSeErR@UusS    -VirtualBox:~$ python3 echo_udp_server.py &
UusSeErReErR     -VirtualBox:~$ python3 echo_udp_client.py
Sent   @UusS: Hello UDP
EchoedeErR: Hello UDP
Sent      : Datagram test
Echoed    : Datagram test
UusSeErR@UusSeErR-VirtualBox:~$ sudo python3 myping.py 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 32 bytes of data.
32 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.015 ms
```

Packet	*Protocol*	*Source*	*Destination*	*Details*
1, 3	UDP	127.0.0.1 : 43740	127.0.0.1 : 6000	Datagram from the client (17 and 21 byte payload)
2,	UDP	127.0.0.1 : 6000	127.0.0.1 : 43740	Echo from the server
4	ICMP	127.0.0.1	127.0.0.1	Type 8 - Echo Request
5, 7	ICMP	127.0.0.1	127.0.0.1	Type 0 - Echo Reply
6, 8				

The capture was taken on the loopback interface, so the MAC addresses are all zeros. On a real interface (enp0s3) the actual MAC addresses are shown.

RESULT:

Thus a packet sniffer was implemented using raw sockets. The Ethernet, IP and UDP / ICMP header details of the packets passing through the interface were captured and displayed successfully.

Exp No: 15	*Analyse various types of servers using Webalizer tool*

AIM:

To analyse the log files of various types of servers (web, proxy and FTP servers) using the Webalizer web server log analysis tool and to study the usage statistics generated by it.

ALGORITHM / PROCEDURE:

1. Install Webalizer in Linux using sudo apt install webalizer apache2.
2. Locate the log files of the servers to be analysed (Apache, Nginx, IIS, FTP, Squid) as given in the table below.
3. Edit the configuration file /etc/webalizer/webalizer.conf and set LogFile, LogType, OutputDir and HostName.
4. Run Webalizer for each server. The option -n gives the host name for the report, -o gives the output folder and -F gives the log type (clf, w3c, ftp, squid).
5. Open the generated usage_YYYYMM.html file in a web browser (http://localhost/webalizer/apache/) to see the report.
6. Study the monthly summary, the daily graph, the top URLs, the top clients, the user agents and the HTTP response codes.
7. Compare the results of the different servers and note down the inference.

Servers and their log formats

Server type	*Log file*	*Log format*	*Webalizer option*
Apache web server	/var/log/apache2/access.log	Common Log Format (CLF)	-F clf
Nginx web server	/var/log/nginx/access.log	CLF compatible (combined)	-F clf
IIS web server	C:\inetpub\logs\LogFiles\W3SVC1\u_ex261008.log	W3C extended	-F w3c
FTP server (vsftpd)	/var/log/xferlog	wu-ftpd xferlog	-F ftp
Squid proxy server	/var/log/squid/access.log	Squid native	-F squid

webalizer.conf (important lines)

```
LogFile         /var/log/apache2/access.log
LogType         clf
OutputDir       /var/www/html/webalizer/apache
HostName        apache-server
PageType        htm*
PageType        php
PageType        cgi
HideSite        localhost
```

Sample log records

```
Apache (CLF):
192.168.1.25 - - [08/Oct/2026:10:41:07 +0530] "GET /index.html HTTP/1.1" 200 4521
192.168.1.12 - - [08/Oct/2026:10:41:09 +0530] "GET /css/style.css HTTP/1.1" 304 -
10.0.2.2 - - [08/Oct/2026:10:42:15 +0530] "GET /missing.html HTTP/1.1" 404 492 IIS
(W3C):
#Fields: date time s-ip cs-method cs-uri-stem sc-status sc-bytes
2026-10-08 05:12:44 192.168.1.30 GET /default.aspx 200 6120
```

DIAGRAM:

Servers	*Log files*	*Analyser*	*Reports generated*
Apache web server	access.log (clf)		
Nginx web server	access.log (clf)		usage_202610.html
IIS web server	u_ex261008.log (w3c)	***WEBALIZER*** -n hostname -o outdir -F logtype	usage_202610.png (graphs)
FTP server (vsftpd)	xferlog (ftp)		webalizer.hist(history)
			Web browser (localhost/webalizer)

Fig 1: Log analysis of different servers using Webalizer

OUTPUT:

```
UusSeErR@UusSeErR-VirtualBox: ~
UusSeErR@UusSeErR-VirtualBox:~$
  r/log/apache2/access.log
Webalizer V2.23-08 (Linux 5.15.0-91-generic) English
Using logfile /var/log/apache2/access.log (clf)
Creating output in /var/www/html/webalizer/apache
Hostname for reports is 'apache-server'
History file not found...
Generating report for October 2026
Generating summary report
Saving history information...
1243 records in 0 seconds
UusSeErR@UusSeErR-VirtualBox:~$
  log/nginx/access.log
Using logfile /var/log/nginx/access.log (clf)
Creating output in /var/www/html/webalizer/nginx
Generating report for October 2026
Generating summary report
Saving history information...
856 records in 0 seconds
UusSeErR@UusSeErR-VirtualBox:~$ sudo webalizer -n iis-server -o /var/www/html/webalizer/iis -F w
  8.log
Using logfile u_ex261008.log (w3c)
Creating output in /var/www/html/webalizer/iis
Generating report for October 2026
Generating summary report
Saving history information...
522 records in 0 seconds
UusSeErR@UusSeErR-VirtualBox:~$ sudo webalizer -n ftp-server -o /var/www/html/webalizer/ftp -F f
  xferlog
Using logfile /var/log/xferlog (ftp)
Generating report for October 2026
64 records in 0 seconds
```

Report of the Apache web server

Monthly Statistics for October 2026	
Total Hits	1243
Total Files	1131
Total Pages	402
Total Visits	138
Total KBytes	18642
Total Unique Sites	57
Avg / Max Hits per Day	155 / 276
Avg / Max Files per Day	141 / 252
Avg / Max Pages per Day	50 / 93
Avg / Max Visits per Day	17 / 23
Avg / Max KBytes per Day	2330 / 3342

Top URLs by hits

#	*Hits*	*%*	*URL*
1	214	17.2 %	/index.html
2	188	15.1 %	/css/style.css
3	163	13.1 %	/images/logo.png
4	97	7.8%	/login.php
5	76	6.1%	/about.html
6	52	4.2%	/contact.html
7	41	3.3%	/favicon.ico

Top clients

#	*Hits*	*%*	*Client (site)*
1	311	25.0 %	192.168.1.25
2	204	16.4 %	192.168.1.12
3	142	11.4 %	10.0.2.2
4	96	7.7%	192.168.1.30

Hits by response code

HTTP response code	*Hits*	*%*
200 - OK	101	82.0 %
304 - Not Modified	9	9.0%
404 - Not Found	112	6.0%
301 - Moved Permanently	74	1.7%
403 - Forbidden	21	0.7%
500 - Internal Server Error	9	0.6%

Comparison of the servers

Server	*Records*	*Total hits*	*Visits*	*KBytes*	*Most requested URL*
Apache	1243	1243	138	18642	/index.html
Nginx	856	856	97	12408	/api/status
IIS	522	522	61	7915	/default.aspx
FTP (vsftpd)	64	64	9	41230	/pub/notes.pdf

Inference

- The Apache server is the busiest server and its highest load was on 08-Oct-2026 (276 hits).
- About 82% of the requests were successful (200) and 6% gave 404 errors, which means some links on the site are broken.
- The home page /index.html is the most visited page. Image and CSS files form most of the hits, whereas only 402 hits are actual pages.
- The same tool can analyse all the servers because only the log type option (-F) changes.

RESULT:

Thus the log files of various types of servers (Apache, Nginx, IIS, FTP) were analysed using the Webalizer tool and the usage statistics such as hits, visits, top URLs, clients and response codes were studied.